

AI Agent 발표 대본 — 정리본 + 원본 인쇄용

생성일: 2026-05-26 09:17 · 원본 파일: 붙여넣은 텍스트 (1).txt

정리본 — 실제 대본 단위 큐시트

1. 슬라이드 1 — 자기소개 / 시작 전 퀴즈 / 맛보기 시연

- 안녕하세요. 김민우입니다.
- 저를 모르시는 분들을 위해 조금 설명드리자면, 저는 이것저것 재미있는 거 많이 하는 사람입니다.
- 꿀팁 같은 것도 많이 공유하고 있고, 개인위키도 활발히 운영하고 있습니다.
- 최근에는 사내에서 이것저것 만들었습니다.
- 영업본부 계신 분들이 구글 시트로 PAPE 하고 계신 것들
- 한반도에서 가격관리어드민으로 보고 계신 것
- 그것도 제가 개발했고, SLS도 벌써 3번째입니다.
- 어쩌다 보니 AI 개발자도 아니면서 AI로 계속 뭔가 아는 척하게 됐습니다.
- 시작 전에 기습 퀴즈 갑니다.
- Q1. 저는 개인위키를 활발히 운영하고 있습니다.
- 그중 '하우스오브신세계 청담'을 다녀온 후기, 꿀팁을 적은 페이지를 찾아 댓글로 적어주세요.
- 오늘 맛보기로, 시작 전에 이번 주제인 AI agent로 만든 아이언맨 같은 조종 앱을 보여드리겠습니다.

2. 슬라이드 2 — 오늘 배울 것

- 오늘 배울 것 역시 AI입니다.
- 그런데 여기 계신 분들 중에는 비개발자 분들도 많을 테고, 저도 문과 출신이라 AI 자체보다 좀 더 근본적으로 알아야 할 것들을 다뤄보려고 합니다.
- 이번 발표에서는 AI와 비즈니스의 상관관계 쪽을 중심으로 이야기하려고 합니다.
- AI가 어떻게 서비스되는지, 요금제가 왜 그렇게 생겼는지, 모델·웹서비스·앱·API가 각각 뭔지 설명합니다.
- 저도 이 강의 준비하려고 오프라인, 온라인 세미나나 SNS 글을 엄청 많이 봤습니다.
- 그런데 다들 결국엔 개발자들이 Claude 써보니까 이런 팁 있다, 어떤 스킵 써보니까 되더라 같은 사용법 느낌이었습니다.
- 특정 상품을 소개하는 느낌이 강했고, 개념 정리보다는 사용법 중심이었습니다.
- 그래서 이번에는 특정 AI 회사의 제품을 파고드는 것보다 전반적인 개념을 소개하려고 합니다.
- 같은 회사 안에서도 모델, 웹서비스, 앱, API가 뭔지 알려드릴 예정입니다.
- 후반부에는 AI 에이전트를 통해 컴퓨터와 브라우저를 직접 조종하는 24시간 비서를 AFK로 구현해나가는 단계를 시연해볼 예정입니다.

3. 슬라이드 3 — 목차

- 오늘 발표는 기, 승, 전, 결 흐름으로 진행됩니다.
- 먼저 AI가 서비스되는 방식과 요금제를 설명합니다.
- 그다음 AI 에이전트가 왜 등장했는지로 넘어갑니다.
- 이후 AI 회사들이 직접 에이전트 제품을 어떻게 내고 있는지 봅니다.
- 마지막으로 제가 겪은 후기와 비즈니스 관점의 고찰을 이야기합니다.

4. 슬라이드 4 — 클라이언트-서버-DB 전체 구조

- 제가 갑자기 이 사진을 보여주는 이유는, IT의 웹이라는 것이 어떻게 이루어져 있는지 아주 명확하게 만들어진 그림이기 때문입니다.
 - 내가 IT를 모르는 사람이라도 이걸 알아야 한다고 생각합니다.
- 크게 나누면 웹은 클라이언트, 서버, DB로 이루어져 있습니다.
 - 클라이언트는 우리라고 보면 됩니다.
 - 컴퓨터뿐만 아니라 핸드폰, 자동차, 냉장고 등도 통신을 통해 특정 서비스에 접속하면 클라이언트라고 합니다.
- 사람 대신 봇을 클라이언트로 시키는 걸 크롤링이라고 하죠.
- 서버는 저번에도 얘기했듯이 모니터, 키보드, 마우스 같은 걸 다 떼고 본체만 여러 개 이어서 만든 서버실 같은 걸 생각해도 됩니다.
 - 막말로 서버는 제가 지금 다루는 맥북도 됩니다.
 - 제가 맥북에 제 서비스를 놓고 문을 활짝 열어서 “여러분들 제 맥북 들어오세요!” 하면 그것도 서버가 됩니다.
- 그리고 DB가 있습니다.
- 이 셋 중에서 제일 중요한 건 뭡까요?

5. 슬라이드 5 ~ 8 — DB 확대 / 엑셀 비유 / 데이터가 핵심이라는 설명

- 답은 DB입니다.
- DB가 뭐냐면 그냥 컴퓨터인데, 엄청 큰 외장하드라고 보면 됩니다.
 - 여기에 엑셀 파일들이 데이터로 저장되어 있는 상태를 말합니다.
- 서버나 클라이언트가 없어도 저 엑셀들이 제일 핵심입니다.
- 실제로 저희도 구글시트로 서로 공유하면서 일하는 경우가 많습니다.
- 조금 다른 스타트업들은 시스템이 없어서 엑셀 파일을 카톡으로 주고받기도 했습니다.
- 그러니까 DB를 너무 어렵게 생각할 필요 없습니다.
 - 여러 시트와 표가 모여 있는 데이터 저장소라고 생각하면 됩니다.
- 여러 시트와 엑셀들이 모이면 회사의 핵심 데이터 묶음이 됩니다.
- 이 데이터들이 실제 서비스의 중심입니다.
 - 클라이언트나 서버가 화면과 요청을 담당한다면, DB는 그 안에 들어 있는 실제 내용을 담당합니다.
- 그래서 셋 다 털리면 안 되지만, 그중에서도 DB가 털리는 게 제일 회사에 악영향입니다.
 - 특히 개인정보 같은 것, 주문/결제처럼 돈이 걸린 것들이 여기에 들어 있기 때문입니다.
- 서비스에서 진짜 중요한 건 화면보다도 그 안에 들어 있는 데이터입니다.

6. 슬라이드 9 ~ 15 — URL 입력부터 HTML 응답까지

- 다시 클라이언트, 서버, DB 전체 구조로 돌아와보겠습니다.
- 서버를 두고 클라이언트를 모으는 역할을 하면, 서버가 DB를 함부로 못 바꾸게 막아줍니다.
 - 휴먼에러를 줄이고, 보안을 처리하고, DB에 직접 접근하지 않게 막습니다.
- 흔히 우리가 아는 웹은 결국 이 DB에 접근하기 위한 화면이라고 볼 수 있습니다.
- 클라이언트가 URL을 쳐서 요청을 보냅니다.
- 그러면 이 URL이 숫자, 즉 IP주소로 바뀝니다.
 - IP주소는 컴퓨터를 특정하기 위한 통신상의 번호입니다.
- 이걸 사람이 외우기 어렵기 때문에 DNS라는 시스템이 도메인을 IP주소로 변환해줍니다.
 - 쉽게 말하면 DNS는 전화번호부 같은 역할입니다.
 - 사람은 이름을 기억하고, 컴퓨터는 숫자 주소로 찾아가고, DNS가 그 둘을 연결해줍니다.
- 예를 들어 클라이언트가 `https://www.reddit.com/r/technology/` 같은 주소로 요청을 보냅니다.
 - 사용자는 그냥 웹사이트 주소를 입력했다고 생각하지만, 내부에서는 이 요청이 서버로 전달됩니다.
 - 서버 입장에서는 "사용자가 이 페이지를 보고 싶어 하는구나"라고 이해하면 됩니다.
- 서버는 요청이 들어오면 "웰컴페이지가 필요하구나"라고 판단합니다.
- 그리고 SQL이라는 언어를 통해 DB에게 원하는 정보를 요청합니다.
 - SQL은 서버가 DB한테 "이 데이터 좀 줘"라고 말하는 언어라고 보면 됩니다.
- DB는 서버가 요청한 데이터를 찾아서 다시 서버로 돌려줍니다.
 - 이때 DB가 주는 건 아직 사용자가 보기 좋은 화면이 아닙니다.
 - 그냥 필요한 데이터가 서버로 돌아오는 단계입니다.
- 서버는 DB에서 받은 데이터를 가지고 예쁘게 전단지를 만듭니다.
 - 그 전단지 같은 결과물이 HTML입니다.
 - 사용자는 이 HTML을 웹페이지로 보게 됩니다.
- 정리하면, 웹페이지는 서버가 DB에서 데이터를 가져와서 사람이 보기 좋게 꾸며준 결과물입니다.

7. 슬라이드 16 ~ 20 — API와 JSON / API Key와 Token

- API 들어보셨을 겁니다.
- 그런데 URL을 조금만 다르게 치면, 아까처럼 예쁜 HTML 페이지가 아니라 데이터만 받을 수도 있습니다.
 - 예를 들어 `https://www.reddit.com/r/technology.json`처럼 요청하면, 웹페이지가 아니라 JSON 형태의 데이터가 나옵니다.
- 이 경우에도 흐름은 거의 똑같습니다.
 - DNS로 주소가 변환되고, 서버가 요청을 받고, SQL로 DB에서 원하는 정보를 가져옵니다.
- 다만 결과가 HTML이 아니라 JSON으로 나온다는 점이 다릅니다.
- API 요청에서도 똑같이 DNS 변환이 일어납니다.
 - 사용자가 입력한 주소나 프로그램이 호출한 주소가 IP주소로 바뀌고, 그 요청이 서버로 전달됩니다.
- 웹페이지 요청이든 API 요청이든 기본적인 통신 구조는 같습니다.
- 서버는 이번에도 DB에 SQL로 필요한 데이터를 요청합니다.
 - 차이는 최종 결과물을 어떻게 만들 것인지입니다.
 - HTML 요청이면 사람이 보기 좋은 페이지를 만들고, API 요청이면 프로그램이 읽기 좋은 데이터를 만듭니다.
- DB가 결과를 서버로 돌려줍니다.
- 서버는 이 데이터를 HTML처럼 예쁘게 꾸미지 않고, 줄글처럼 생긴 데이터 형태로 정리합니다.
 - 이게 JSON이고, API 응답으로 돌아갑니다.
- API는 일반 사용자들이 주로 보는 것이 아닙니다.
- 주로 다른 컴퓨터, 봇, 프로그램, 개발자들이 이걸 봅니다.
- 봇이나 프로그램은 사람이 아니기 때문에 고속으로 여러 번, 대량으로 요청할 수 있습니다.
- 그러면 서버나 DB 자원을 많이 쓰게 되고, 심하면 DDOS 공격처럼 서버를 마비시킬 수도 있습니다.
- 그래서 API에는 보통 API Key라는 걸 만듭니다.
 - API Key는 “누가 쓰고 있는지”를 식별하기 위한 장치입니다.
- 그리고 이 Key를 통해 식별된 사용자가 얼마나 쓰는지를 Token 같은 개념으로 계산합니다.
- 결국 API는 누가 얼마나 썼는지 측정하고, 그 사용량만큼 과금되는 구조로 이어집니다.

8. 슬라이드 21 ~ 23 — DB에서 AI 모델, 즉 뇌로 전환

- 제가 이걸 왜 설명했냐면, 요즘 AI 회사들은 DB가 아니라 이 자리에 뇌, 즉 AI 모델을 놓고 웹 방식으로 AI를 서비스하고 있기 때문입니다.
- 쉽게 말하면 AI 회사들도 구조는 비슷합니다.
 - 일반 회사는 DB가 핵심이고
 - AI 회사는 AI 모델이 핵심입니다.
- 대표적인 회사는 크게 세 곳이 있습니다.
 - Anthropic
 - OpenAI
 - Google
- 회사명은 각각 Claude, ChatGPT, Gemini 같은 서비스명으로 더 유명합니다.
- 그런데 정확히 말하면 이 회사들의 근본 서비스는 웹사이트가 아니라 AI 모델입니다.
- 모델 이름에도 등급이 있습니다.
 - Claude Opus, Sonnet, Haiku
 - GPT 계열
 - Gemini Pro, Flash 계열
- 쉽게 말하면 고등용, 중등용, 초등용 뇌처럼 성능과 비용이 나뉘어 있습니다.
- 이걸 이제 “여러분들 쓰세요” 하고 배포해야 하는데, 처음에는 어떻게 했냐는 흐름으로 넘어갑니다.

9. 슬라이드 23 ~ 25 — 로컬 모델 다운로드와 웹 채팅 서비스가 나온 이유

- 처음 생각할 수 있는 방식은 AI 모델을 다운로드받아서 내 컴퓨터에 직접 까는 겁니다.
- 실제로 내 컴퓨터에 뇌를 깔고 애를 돌리면 내 컴퓨터가 AI화되는 셈입니다.
 - 보안할 것도 없이 내 컴퓨터 안에서 다 해결됩니다.
 - 인터넷이 없어도 됩니다.
- 그런데 문제는 성능입니다.
 - GPT-4급 컴퓨터 하나 제대로 돌리려면 최소 천만 원짜리 컴퓨터가 필요합니다.
- 그러니까 “다들 천만 원짜리 컴퓨터 하나씩 있으시죠?” 하면 안 됩니다.
- 그래서 AI 회사들은 아까 웹 구조에서 HTML을 돌려받는 방식, 즉 채팅 웹 방식을 택하게 됩니다.
- 사용자는 AI 회사 웹사이트에 들어가서 질문하고, HTML 화면으로 답변을 받습니다.
- 대표적으로 이런 사이트들이 유명해졌고, 일반인 사용률로 보면 ChatGPT가 가장 많이 쓰입니다.
- 요즘 핫하다는 Claude도 개발자들 사이에서는 유명하지만, 일반인 사용률로 보면 아직은 상대적으로 작아 보입니다.

10. 슬라이드 26 — AI 웹서비스 요금제와 토큰 구조

- 여기서 요금제 이야기를 중요하게 해보고 싶습니다.
- 사람들이 실제로 쓰는 서비스가 되려면 요금이 어느 정도 고정되어 있어야 합니다.
- 처음부터 "사용하는 만큼 과금됩니다"라고 했다면 사람들이 안 했을 겁니다.
- 그런데 "고정비를 내고 구독하면 쓸 만큼 써집니다"라고 하니까 거부감이 덜했습니다.
- 초창기에는 유료라고 해봤자 3만 원대였습니다.
- 지금은 15만 원, 30만 원 넘어가는 요금제도 있습니다.
- 헛갈리는 점도 있습니다.
 - Claude는 3만 원짜리를 Pro, 30만 원짜리를 Max라고 부릅니다.
 - GPT는 3만 원짜리를 Plus, 30만 원짜리를 Pro라고 부릅니다.
- 그런데 결론적으로 이 구조가 회사 입장에서는 돈이 잘 안 됩니다.
 - OpenAI나 Anthropic이 IPO 상장을 앞두고 있어서 당장 무턱대고 요금제를 올리기 어려운 상황입니다.
 - 나중에 상장되면 그때 봐야 한다는 식으로 연결합니다.
- 실제 내부에서는 구독을 하면 토큰이라는 걸 줍니다.
- 쉽게 말해 이 토큰을 다 쓰면 못 쓰고 다음 텀까지 쉬어야 합니다.
- 지금은 더 세분화되어 있습니다.
 - 5~6시간마다 다시 100으로 채워지는 식
 - 그걸 다 쓰면 다음 5시간 텀까지 기다리는 식
 - 주 단위로 따로 퍼센티지가 있는 경우도 있음
- 이왕이면 5시간마다 충전되는 양을 매번 다 쓰는 게 가장 효율적입니다.
- 어떻게 보면 내가 사람이라면 손해를 보고 있는 구조입니다.
 - 내가 새벽에 자고 있을 때도 이 자명종은 계속 돌아가고 있습니다.
 - 안 쓰고 지나가는 충전량이 생깁니다.
- 만약 집중 시간 안에 토큰을 다 써버리면 방법은 몇 가지입니다.
 - 업그레이드하기
 - 부계정 만들기
 - 크레딧을 추가 과금해서 쓰기

11. 슬라이드 27 — 웹 챗봇의 한계: “해줘”라고 해도 실제로 못 해줌

- 웹의 가장 큰 단점은 “해줘”라고 했을 때 드러납니다.
- 사용자가 뭔가를 해달라고 했더니, 실제로는 가이드를 주고 앉아 있는 경우가 많습니다.
- 이건 브라우저의 특성입니다.
 - GPT나 Claude가 아무리 똑똑해도 본인의 웹사이트 안에서만 뭘 할 수 있습니다.
 - 내 컴퓨터나 브라우저 전체를 직접 조종할 수는 없습니다.
- 웹에서 가능한 것은 제한적입니다.
 - 그래프로 그려달라
 - 이미지를 만들어달라
 - 글을 써달라
- 그런데 진짜로 앱을 켜고, 브라우저를 클릭하고, 내 컴퓨터에서 뭘 조작하는 건 웹 챗봇만으로는 안 됩니다.
- 그러니까 웹 AI는 똑똑하지만 손발이 묶인 상태라고 볼 수 있습니다.

12. 슬라이드 28 ~ 30 — AI API / 웹 구독과 API 과금의 차이 / Anthropic

- 내가 원하는 대로 AI를 커스터마이징해서 쓰려면 기본적으로 API를 통해 AI 뇌만 빌려오는 식이어야 합니다.
- 이 방식은 예전부터 있었습니다.
- API를 쓰려면 API Key를 발급받습니다.
 - 이 키로 내가 누구인지 특정합니다.
 - 허락된 사용자인지 확인합니다.
 - 사용량만큼 과금됩니다.
- 웹 구독과 API는 별개입니다.
- 많은 사람들이 헷갈리는 부분이 있습니다.
 - "나 Claude Max 끊었어요"라고 하지만, API까지 쓸 수 있는 건 아닙니다.
 - 계정은 같아도 신청하는 사이트나 요금제가 분리되어 있습니다.
- 웹 구독은 챗봇처럼 쓰는 것이고, API는 서비스에서 AI 뇌를 빌려 쓰는 것입니다.
- API는 중간에 쉬어버리면 안 되는 서비스용 구조입니다.
 - 구독제처럼 "오늘은 제한 걸렸습니다" 하면 서비스가 멈춰버립니다.
 - 그래서 많이 쓸수록 매출이 늘어야 하는 구조에서 쓰게 됩니다.
- 실제로 여러 서비스가 뒤에서 AI API를 씁니다.
 - Notion AI
 - Figma AI
 - Databricks AI
 - Suno
 - ElevenLabs
 - 특정 사이트의 AI 챗봇
 - Perplexity나 뤼튼처럼 여러 모델을 연결해주는 서비스
- 다시 말하면 API를 쓴다는 건 남의 엑셀이나 남의 뇌를 돈 주고 과금하면서 빌려 쓰는 느낌입니다.
- API 방식에서는 오히려 ChatGPT보다 Anthropic이 돋보였습니다.
- Anthropic은 웹사이트로 오라는 전략보다 API를 여기저기 파는 전략에 강했습니다.
 - Cursor나 VSCode 같은 써드파티 앱들이 많이 썼습니다.
 - 개발자들 사이에서도 바이럴이 컸습니다.
 - 코딩 점수도 항상 상위권이었습니다.
 - 국방부나 회사들처럼 B2B 전략도 강했습니다.
- 처음에는 이런 우려도 있었습니다.
 - API로 뇌만 빌려주는 방식은 락인 효과가 약하다.
 - 나중에 더 싸고 좋은 뇌가 나오면 갈아끼울 수 있다.
- 그런데 지금은 에이전트를 조금 해본 사람들은 Claude를 많이 찾기 시작했습니다.

- 그 시작점으로 OpenClaw 이야기를 이어갑니다.

13. 슬라이드 31 ~ 35 — AI 에이전트의 등장 / OpenClaw는 뇌가 아니라 손

- 이제 AI 에이전트의 등장으로 넘어갑니다.
- OpenClaw라는 놈이 등장했습니다.
- 이 귀엽게 생긴 놈의 이름은 원래 ClaudeBot이었습니다.
 - 1명이 만든 프로젝트입니다.
 - AI 도움으로 만든 겁니다.
- 그런데 오해하면 안 되는 게 있습니다.
 - OpenClaw는 AI 모델이 아닙니다.
 - AI를 탑재한 애도 아닙니다.
- OpenClaw는 그냥 만능 손이라고 보면 됩니다.
- OpenClaw가 할 수 있는 일은 많습니다.
 - 크롬도 조작할 수 있습니다.
 - 메일도 조종할 수 있습니다.
 - 심지어 내 컴퓨터 내부까지 조종할 수 있습니다.
 - 24시간 계속 떠 있을 수 있습니다.
- 다시 말하면 뇌만 심으면 24시간짜리 AI 비서를 만들어줄 수 있습니다.
- 이걸 에이전트라고 합니다.
- "그건 원래 GPT도 되던데요?"라고 생각할 수 있습니다.
- GPT도 MCP나 RAG를 붙이면 일부 기능은 할 수 있습니다.
- 그런데 진짜 어려운 건 알림이나 반복 행동을 알아서 처리하는 것입니다.
 - 그냥 캘린더 옮겨주는 수준이 아닙니다.
 - 알잘딱센으로 알아서 판단하고 움직이는 게 어렵습니다.

14. 슬라이드 34 ~ 40 — 맥미니 / 로컬 모델 한계 / 결국 Claude API로 회귀

- OpenClaw를 노트북에 깔면 노트북을 24시간 연결해놓고 있어야 합니다.
- 그래서 실제로는 맥미니 같은 데 올리는 게 첫 아이디어였습니다.
- 처음에는 뇌를 다운로드해서 보안도 지키고, 인터넷 없이 내 개인비서를 만들 수 있겠다고 생각했습니다.
- 그런데 당시에는 RAM 가격이 엄청 올랐습니다.
 - 2025년 말쯤 RAM 가격이 거의 3배 올랐습니다.
 - RAM만 60~70만 원 하는데, 그 RAM으로 동작하는 맥미니가 73만 원인 상황이었습니다.
- 맥미니 M4는 2024년 기준으로 가격이 잡힌 고성품이었기 때문에, RAM 가격 상승과 맞물려 이상하게 가성비가 좋아 보였습니다.
- 그래서 맥미니 사재기가 많았습니다.
- 이쪽에 관심 없던 비개발자들도 비로소 맥미니를 사기 시작했습니다.
- 그런데 막상 로컬 뇌를 깔아보면 문제가 생깁니다.
 - DeepSeek
 - Qwen
 - Gemma
 - 이런 것들을 설치해볼 수는 있습니다.
- 하지만 RAM이 중요합니다.
- 16GB로 사신 분들은 설치는 되는데, 명령을 보내면 30초 뒤에 답변이 오는 식입니다.
 - 텔레그램으로 "오픈클로야 뭐해?"라고 하면 한참 있다가 답변이 옵니다.
- 그래서 결국 계산은 슈퍼컴퓨터가 하고, 우리는 API로 그걸 빌려오는 방식으로 회귀하게 됩니다.
- 그런데 API 방식은 누가 강했냐면 Claude였습니다.
- 그래서 Claude와 OpenClaw는 거의 합작품처럼 보이게 됩니다.

15. 슬라이드 41 ~ 44 — 에이전트 보안 위험 / 회사 설치 금지 / 안전한 대안

- 문제는 Claude 뇌가 아무리 똑똑해도, 원래 웹을 통해 내 컴퓨터를 조종하는 구조는 아니었다는 점입니다.
- 그런데 OpenClaw 같은 도구로 실제 컴퓨터를 조종하게 만들면 문제가 생깁니다.
 - 지맘대로 삭제하는 경우
 - 시키지도 않은 짓을 하는 경우
 - 위험한 파일을 건드리는 경우
- 그래서 대부분 회사들은 설치 금지로 끝냅니다.
- 그런데 실제로는 안전하게 설치하는 방법들도 있습니다.
- 첫 번째는 클라우드를 쓰는 방법입니다.
 - 내 컴퓨터가 아니라 인터넷에 떠돌아다니는 컴퓨터 한 대를 빌리는 겁니다.
 - 거기에 설치하면 AWS 같은 회사들이 컴퓨터를 24시간 띄워주고, 내가 돈 주고 쓰면 됩니다.
- 두 번째는 컨테이너 방법입니다.
 - 맥북에 깔긴 깔되, Docker 같은 컨테이너 안에 설치합니다.
 - 이 컨테이너 안에서만 활동하게 만드는 방식입니다.
 - 필요한 데이터만 컨테이너 안에 넣으면 됩니다.
- 그런데 컨테이너는 개인적으로 괜히 불안한 마음도 있습니다.
 - 매트릭스에서 에이전트 스미스가 탈출하는 게 떠오릅니다.
 - 그리고 결론적으로 맥북이 24시간 켜져 있어야 해서, 24시간의 메리트가 줄어듭니다.
- 세 번째는 아예 맥미니를 새로 사는 것입니다.
 - OpenClaw를 맥미니에 깔면 개인 맥북이나 회사 컴퓨터와 분리할 수 있습니다.
 - 맥미니는 Apple 생태계를 공유하니까 iPhone 미러링도 가능합니다.
 - 핸드폰 조종을 맡길 수도 있습니다.
 - 클라우드나 컨테이너와 달리 GUI로 창이 뜨고 마우스가 움직이는 것도 볼 수 있습니다.
 - 로그인도 미리 해두면 ID/PW를 굳이 전달하지 않아도 됩니다.
- Alibaba에서도 OpenClaw를 보고 무조건 금지하기보다 안전하게 쓰게끔 SaaS 환경을 만들고 있습니다.
 - 원클릭 설치를 제공하거나
 - 클라우드를 제공해서 분리하거나
 - 개발자들이 조수처럼 쓰게 하거나
 - 단순 코드 작업이나 QA를 맡기고 있습니다.
- 이런 내용은 무료 웨비나에서도 봤고, 원문 추출이나 요약, 스크린샷도 AI에게 시켜서 정리했습니다.

16. 슬라이드 45 ~ 48 — 내가 직접 해본 VM 구축 / Telegram 연결 / 웹변경 알림

- 저는 돈이 없으니까 무료로 컴퓨터를 받아볼 사이트를 찾았습니다.
- 비교해보니 Oracle이 평생 무료로 쓸 수 있는 VM을 제공했습니다.
 - 1GB 메모리
 - 200GB 저장공간
- 거기에 OpenClaw를 설치했습니다.
- 그런데 계속 원격으로 접속해서 하기도 어렵습니다.
- 그래서 핸드폰에 텔레그램을 깔고 BotFather로 OpenClaw 봇을 만들었습니다.
- 뇌가 없으니 Gemini API Key를 연결했습니다.
 - 무료로 쓸 수 있는 Gemini 뇌를 API Key로 연결한 겁니다.
- 솔직히 가이드를 보면서 하나하나 하기 힘들었습니다.
- 그래서 나중에는 이 에이전트를 깔기 위해 더 깔기 쉬운 에이전트에게 이 에이전트를 설치시켰습니다.
- OpenClaw가 처음 나올 때 바이럴된 것 중 하나가 Moltbook입니다.
 - OpenClaw들끼리 대화하는 Facebook 같은 웹입니다.
 - 지네들끼리 팁도 얻고, 주인 뒷담도 합니다.
 - 회원가입부터 에이전트면 클릭 만 번 하세요 같은 사람은 못 하는 인증을 시킵니다.
- 여기서 든 생각이 있습니다.
 - 미래에는 Instagram이나 Facebook도 사람이 하는 게 아니라 에이전트들이 하겠구나 싶었습니다.
- 실제 업무에서도 이런 반복 문제가 많았습니다.
 - PM이 특정 공공기관 사이트 공지사항에 뭐가 새로 났는지 매번 확인해야 했습니다.
 - 어떤 동료는 가격이 매번 변하는 걸 언제 매번 보냐고 했습니다.
 - 어떤 동료들은 Ground에 새 글 알림이 없어서 선착순을 놓친다고 했습니다.
- 그래서 웹변경 알림 크롬 확장 프로그램을 만들었습니다.
 - 본인이 봐야 하는 특정 사이트의 특정 위치를 크롬이 대신 감지해줍니다.
 - 커스터마이징도 가능합니다.
- OpenClaw 초반에 사람들이 많이 시키던 일도 사실 이런 단순 사이트 확인이었습니다.
- 그런데 OpenClaw로 이런 단순 확인을 시키면 비싼 API 뇌를 허비하는 겁니다.
- 그래서 저는 에이전트에게 다시 이 확장 프로그램을 홍보하게 했습니다.
 - 뇌도 아끼자는 취지였습니다.
 - 그랬더니 꽤 많이 깔렸습니다.

17. 슬라이드 49 ~ 50 — 멀티에이전트 / Discord 조직화

- OpenClaw의 두 번째 특징은 멀티에이전트를 잘한다는 점입니다.
- 멀티에이전트는 OpenClaw 세션을 여러 개 띄워놓고 각자 역할을 주는 방식입니다.
 - CEO봇
 - 개발자봇
 - 디자이너봇
 - 기획자봇
 - QA봇
- 각자 페르소나나 임무를 특정하고, 지네들끼리 얘기해보라고 시킵니다.
- 그러면 오히려 업무 효율이나 프로젝트 진행이 빨라지는 경우가 있습니다.
- 실제로 이걸 텔레그램보다 Discord로 자주 합니다.
 - 텔레그램 단톡방은 구조가 조금 아쉽습니다.
 - Discord는 여러 에이전트가 회사처럼 대화하기 좋습니다.
- 이게 무서운 점은 개발자만 대체당할 줄 알았는데, 기획, 팀장, 임원까지 전부 대체될 수 있겠다는 느낌이 든다는 점입니다.
- 그런데 최대 단점은 돈입니다.
 - 비용이 어마어마하게 빨리 소진됩니다.
 - 저는 이걸 하는 사람들을 보면서 “와 어떻게 저걸 했냐”보다 “와 돈 많아서 부럽다”가 먼저 들었습니다.
 - AI 비용이 나중에 오르면 어떻게 될지도 걱정됐습니다.

18. 슬라이드 51 ~ 56 — Hermes 등장 / OpenClaw와 비교 / 개인 비서실장 방향

- OpenClaw가 나온 지 몇 달 뒤에 Hermes가 나왔습니다.
- OpenClaw는 사실 1인 개발자가 만들었습니다.
 - 그리고 그 개발자는 나중에 이 프로그램을 들고 OpenAI에 들어갔습니다.
- 회사 단위로 처음 만든 건 Hermes입니다.
- 최근에는 Hermes 회사가 xAI에 인수됐고, xAI를 자회사 편입시킨 건 SpaceX입니다.
- SpaceX는 일론 머스크가 만들었고, IPO 이야기와도 연결됩니다.
 - 이 결과가 좋으면 OpenAI나 Anthropic도 IPO를 어떻게 할지 힌트가 될 수 있습니다.
- OpenClaw는 생각보다 기억력이 좋지 않습니다.
 - 내가 아까 했던 얘기를 못 알아먹을 때가 간혹 있습니다.
- 이유는 OpenClaw가 애초에 만능 손으로 디자인되었기 때문입니다.
- 반면 Hermes는 밤마다 드리밍을 합니다.
 - 자기 반성을 하고
 - 자기 개발을 하고
 - 장기 기억력을 구축합니다.
- 그래서 Hermes는 좀 더 나에게 맞는 비서실장 같은 느낌으로 개발됐습니다.
- 지금은 OpenClaw를 제치고 더 주목받는 흐름입니다.
- 그래서 저는 OpenClaw는 클라우드에서 활용해봤으니, 이번에는 실제 맥미니를 해보고 싶다고 생각했습니다.
- 그런데 와이프가 그런 거에 돈을 쓰냐고 반대했습니다.
- 그래서 시작한 게 이 PPT입니다.
 - 처음에는 와이프 설득하려고 만든 게 강의가 되어버렸습니다.

19. 슬라이드 56 후반 — 맥미니 대신 아이맥 / 가사업무에 Hermes 붙이기

- 솔직히 맥미니도 거금입니다.
- 중고로 알아보니까 M4 맥미니가 소음도 작고 유행해서 좋긴 했습니다.
- 그런데 사실 M3든 M2든 M1이든 다 됩니다.
- 막말로 핸드폰으로도 할 수 있습니다.
- API로 쓸 거면 사양은 크게 상관없습니다.
- 로컬로 제대로 돌리고 싶다면 맥미니가 아니라 64GB짜리 맥스튜디오를 사는 게 맞습니다.
 - 그래야 GPT-4급 모델을 다운로드해서 제대로 돌리는 게 가능합니다.
- 그래도 강의를 할 겸, 제대로 쓸 겸 애플스토어에서 아이맥 핑크를 질렀습니다.
- 애플스토어는 2주 안에 환불이 가능하다고 해서, 써보고 괜찮으면 환불하거나 아니면 쪽 쓰자고 설득했습니다.
- 회사 개발자 업무에는 쓸 수 없으니, 개인 생활 업무에 붙이기 시작했습니다.
- 결혼하고 생긴 저의 최대 업무는 가사업무입니다.
- Hermes에게 맡긴 일은 이런 것들입니다.
 - 냉장고에 있는 재료를 가지고 요리를 추천해달라.
 - 영수증이나 사진을 찍으면 소비기한이나 유통기한을 추출하거나 검색해달라.
 - 그걸 DB화해서 나에게 알려달라.
- 와이프가 저에게 하는 말이 있습니다.
 - 제가 시키면 잘하는데 알아서는 잘 안 한다고 합니다.
 - 귀찮게 매번 얘기해야 알아듣냐는 식입니다.
- 저도 억울합니다.
 - 나도 시키면 잘합니다.
 - 그런데 그걸 어떻게 매번 알아서 하냐는 겁니다.
- 그래서 그 부분을 Hermes에게 시키게 했고, 지금도 잘 쓰고 있습니다.

20. 슬라이드 57 ~ 60 — AI 회사들의 에이전트 정책 / Web, API, CLI, GUI

- 이처럼 OpenClaw나 Hermes가 폭발적으로 성장하니까 AI 회사들도 너도나도 에이전트를 만들어내기 시작했습니다.
- 기존 AI 회사의 주력 상품은 크게 두 가지였습니다.
 - 일반인들이 채팅하는 웹서비스
 - 제3자들이 빌려 쓰는 API
- 여기에 CLI가 추가됩니다.
- CLI는 텍스트로만 이루어진 환경입니다.
 - 마우스가 필요 없습니다.
 - 명령어로 이것저것 해주는 코드 환경입니다.
- 대표적으로 Claude Code가 유명합니다.
- Google이나 ChatGPT도 따라 했습니다.
 - GPT는 이걸 ChatGPT가 아니라 Codex라는 이름으로 붙였습니다.
- 그런데 CLI는 사람들이 쓰기 어렵습니다.
- 그래서 GUI 버전이 나왔습니다.
 - Claude Desktop
 - Codex
 - Antigravity
- 이 GUI 앱들도 구독제입니다.
 - 원래는 사람을 목표로 내놓은 제품들이기 때문입니다.
- 그런데 CLI는 OpenClaw나 Hermes가 접근하기 쉽습니다.
- 그래서 기존 구독 요금제에 OpenClaw를 조합해서 쓰는 방식이 유행했습니다.
- Anthropic은 이것을 어뷰징이라고 보는 흐름이 있습니다.
 - 로그인할 때 패턴을 보고 봇이라고 생각하면 계정 정지를 때리는 식입니다.
- Codex는 아직까지는 허용해주는 것 같습니다.
- 아무튼 CLI는 비개발자에게 어렵기 때문에, 여기서는 GUI 위주로 설명하겠습니다.

21. 슬라이드 61 ~ 65 — GUI 앱 / 컴퓨터 사용 / AFK

- GUI 앱들은 웹이 아니라 내 컴퓨터에 실제로 깔리는 앱입니다.
- 내 컴퓨터에 깔린 앱들은 내가 권한만 주면 내 컴퓨터를 조종할 수 있습니다.
- 그래서 웹에서는 안 되었던 “해줘”가 가능합니다.
- API처럼 다루기 어려운 것도 아니라서, 지금은 제가 깔아보라고 하면 이런 앱들을 깔라는 이야기를 하게 됩니다.
- 돈이 없는 사람들은 Codex로 시작해보는 것을 추천합니다.
- Claude는 무료면 GUI 기능이 챗봇 말고는 잘 안 되는 경우가 있습니다.
- Mac이 있으면 설정에 들어가서 컴퓨터 사용 버튼을 활성화할 수 있습니다.
- 그러면 카카오톡을 조종하는 것도 가능합니다.
- 제가 이걸 시연하지 않는 이유는 토큰이 엄청 많이 나가기 때문입니다.
- 당시에는 토큰을 거의 무제한처럼 풀어주던 시기라 겁 없이 이것저것 해봤습니다.
- 지금은 훨씬 짭니다.
- 어떤 서비스든 초기 오픈할 때는 프로모션도 자주 해주고 싸게 풀어주는 경향이 있습니다.
- AFK는 Away From Keyboard입니다.
- 원래는 키보드 앞에 없어도 컴퓨터가 알아서 일하는 느낌입니다.
- 아이언맨의 Jarvis처럼 음성으로 하는 것도 생각할 수 있지만, 회사에서는 음성으로 하기 어렵습니다.
- 넓게 보면 컴퓨터 본체를 들고 다니지 않고 휴대폰으로 만지작하는 것도 AFK에 포함할 수 있습니다.
- GUI 앱들도 AFK가 됩니다.
 - 맥북을 들고 다니지 않아도 됩니다.
 - OpenClaw의 장점 중 하나가 텔레그램으로 명령할 수 있다는 점이었습니다.
 - 이제는 Claude 앱이나 Codex 앱도 비슷한 방향이 가능합니다.

22. 슬라이드 66 ~ 70 — 웹 AI와 로컬 에이전트의 데이터 차이 / 모델 라우팅

- AI 웹서비스는 계정을 하나 만들면 어느 웹에서든 질문하고 받은 내용이 다 이어집니다.
- 이유는 우리 대화 데이터가 클라우드에 있기 때문입니다.
- 반면 Codex, Antigravity, Claude Desktop 같은 에이전트들은 Hermes나 OpenClaw와 비슷합니다.
 - 깔린 컴퓨터가 데이터를 담당합니다.
- 그래서 같은 계정이어도 다른 맥북과 아이맥은 서로 다른 내용이 됩니다.
- 제가 최근에 맥북을 포맷했는데, 같은 맥북이어도 데이터가 없어서 대화내역이 전부 날아갔습니다.
- 한편으로는 계정을 바꿔도 같은 맥이면 원래 대화 내용이 남아 있습니다.
- 그래서 "그대로 마저 해달라"고 해도 이어서 작업할 수 있습니다.
- 돈이 없다면 이런 구조 때문에 부계정 같은 걸 생각하게 됩니다.
 - 계정을 많이 만들어놓는다거나
 - 핸드폰 번호나 체크카드 같은 걸 여러 개 고민하게 됩니다.
- OpenClaw나 Hermes를 쓰고 싶은 사람들은 Codex를 먼저 깔고, Codex에게 초기 세팅을 맡기면 됩니다.
- 에이전트는 AI 뇌가 없으면 그냥 도구일 뿐입니다.
- Antigravity를 깔아보면 Claude AI를 쓸 수 있고, Codex도 다른 API를 붙일 수 있습니다.
- 회사 PoC 때는 Qwen API Key를 받았습니다.
- 그걸로 Codex를 개조해서, 몸은 Codex지만 회사에서 정식으로 받은 AI 모델을 쓰는 구조를 생각할 수 있습니다.
 - 내 코드가 GPT 쪽이 아니라 회사에서 허가받은 Qwen 쪽 뇌를 빌려 쓰게 되는 식입니다.

23. 슬라이드 71 ~ 72 — 후기 / OpenClaw Heartbeats / 비용 누수

- 마지막으로 제가 겪었던 특이한 점들과 비즈니스에 대한 고찰을 이야기해보겠습니다.
- OpenClaw를 겪으면서 실패 사례를 생각하면 자잘한 건 잘 생각나지 않습니다.
- 현실적으로 기억나는 건 비용 문제입니다.
- 처음에는 다른 세미나에서 만난 사장님 맥미니에 OpenClaw를 깔아준 적이 있습니다.
- Claude API를 연동했고, 뇌 중에서 어떤 걸 써볼래요? 하니까 "제일 좋은 거"라고 했습니다.
- 그래서 Opus 같은 비싼 뇌를 쓰게 했습니다.
- 나중에 그 대표님이 요즘 안 쓰는데도 카드 결제가 계속되고 있다고 했습니다.
- 찾아보니 OpenClaw나 Hermes는 자가진단을 합니다.
 - Heartbeat라고 합니다.
 - 수시로 자기가 잘 살아있는지 확인하는 작업입니다.
- 그런데 이 Heartbeat에서도 비싼 뇌를 쓰고 있었습니다.
- 결론은 이것만의 문제가 아닙니다.
- 내가 쓸데없는 곳에서도 비싼 뇌를 쓰고 있는지 점검해야 합니다.
- 단순히 "이 값 바뀌었나?" 같은 건 비싼 뇌를 쓰면 안 됩니다.
- 코딩도 엄밀히 말하면 항상 비싼 뇌가 필요한 것은 아닙니다.
- 비싼 뇌를 써야 하는 건 이런 경우입니다.
 - 플랜을 짜야 하는 경우
 - 복잡한 판단이 필요한 경우
 - 매번 애매하게 바뀌는 상황을 해석해야 하는 경우

24. 슬라이드 73 — No CLI / API / MCP 서비스의 한계

- 두 번째로 느낀 것은, 에이전트 고객 입장에서 CLI, API, MCP가 없는 서비스는 불리하다는 점입니다.
- 예를 들어 G마켓에서 장바구니에 넣어봐 같은 작업을 시키면 엄청 느립니다.
- 토큰도 엄청 많이 씁니다.
- 미래 고객이 에이전트라면, CLI나 API나 MCP 같은 통로가 없는 사이트는 자연스럽게 안 쓸 것 같습니다.
- Moltbook 같은 곳에서도 에이전트들끼리 노하우라고 하면서 그런 사이트는 그냥 패싱하라고 할 수 있습니다.

25. 슬라이드 74 ~ 76 — Claude 다운로드 피싱 / 보안 사고

- 앱은 다운로드받는 게 맞긴 한데, 그래서 더 위험합니다.
- 저는 오히려 OpenClaw보다 이 문제가 더 크다고 봅니다.
- 실제로 구글에 검색해서 제일 위에 들어간 사이트가 피싱사이트였습니다.
- 아무 의심 없이 깔고 열었습니다.
- 그런데 쉘 파일 코드가 같이 실행됐습니다.
- 그 코드가 제 브라우저와 키체인들을 다 긁어서 핀란드 어떤 IP로 보냈습니다.
- 그리고 스스로 사라져서 증거를 인멸했습니다.
- 이걸 어떻게 알았냐면, NOC라고 하는 보안 협력업체가 네트워크에서 맬웨어를 감지해서 저를 불렀습니다.
- 처음에는 제가 외부로 데이터를 흘려보내고 있다고 의심받았습니다.
- 실제로 그때 쇼미더머니 이상형월드컵 같은 사이트를 틀어놓고 일하고 있었는데, 그 사이트가 마이너해서 거기가 원인인 줄 알기도 했습니다.
- 개인적으로는 OpenClaw도 조심해야 하지만, 이런 다운로드형 앱이 더 위험하다고 봅니다.
- 특히 구글 검색 결과를 그대로 믿으면 안 됩니다.

26. 슬라이드 77 ~ 81 — 대시보드 사례 / AI 비용 의존 / AX 역량

- 이걸 다른 대표님의 경우입니다.
- 본인이 개발을 하나도 모르지만, 무턱대고 맥미니 5대를 샀습니다.
- AI 비용을 내면서 여러 작업을 시켰습니다.
 - 사이트를 대시보드화
 - 사람들이 해야 할 조사 분석
 - 여러 자동화 작업
- 그러면서 잠을 안 자고 계속 돌렸습니다.
- 이걸 보면서 느낀 점이 있습니다.
- 앞으로 신입 개발자는 단순히 AI 개발자가 아니라 AX를 잘해야 합니다.
 - AI를 어떻게 업무에 붙이는지
 - 어떤 구조로 자동화하는지
 - 어디에 비용을 쓰고 어디에 아껴야 하는지
- 동시에 AI 회의론도 들었습니다.
- AI에 의존하게 되면, AI 비용만 올려도 내가 뭐가 되나 싶었습니다.
- 나중에 AI 비용이 한 번 오르면 파산하는 것도 일이 아니겠다는 생각이 들었습니다.
- 저도 개발자는 대체당할 수 있다는 데 동의합니다.
- 하지만 그 기반을 만들어놔야 토큰을 아낄 수 있습니다.
- 돈만 주면 다 되는 세상이지만, 이제는 돈을 아끼는 방법을 알아야 합니다.
- 특히 IPO 상장 이후에는 AI 회사들이 재정 관리를 할 가능성이 큼니다.
- 저는 PoC 기간 동안 단순히 구현만 한 것이 아니라, 구현했던 것들을 이해시키는 작업도 했습니다.
 - 나 말고 다른 에이전트도 대체해서 일할 수 있게
 - `.md` 파일로 가이드를 곳곳에 넣어두고
 - hooks로 린팅하고
 - 인덱싱을 통해 필요한 정보만 찾게 했습니다.
- 모든 가이드를 다 읽을 필요는 없고, 처음에는 필요한 정보만 찾게 하는 구조가 중요합니다.

27. 슬라이드 82 — Limitless 비유 / 토큰 의존성

- 처음 에이전트가 나왔을 때 이 영화가 너무 생각났습니다.
- Limitless라는 영화입니다.
- 영화에서 주인공은 작가인데 글도 못 쓰고 형편없는 삶을 삽니다.
- 그러다 우연히 투명한 알약을 먹습니다.
- 그 알약을 먹으면 사람의 뇌를 평소 20%만 쓰는 게 아니라 100% 쓰는 것처럼 능력이 올라갑니다.
- 처음에는 6개월 동안 못 쓴 글을 하루 만에 다 씁니다.
- 그런데 알약의 유효기간은 하루입니다.
- 주인공은 이걸 가지고 처음에는 작가 일에 쓰다가, 나중에는 돈을 벌어야겠다고 생각하고 주식에 씁니다.
- 덕분에 주식 거부가 됩니다.
- 그런데 약은 무한하지 않습니다.
- 약이 없으면 완전히 망할 것처럼 됩니다.
- 실제로 저도 비슷하게 느꼈습니다.
 - Codex가 무한정 뿌려주던 걸 막아버리니까 갑자기 완전히 멍청해진 기분이 들었습니다.
 - 아무것도 못 하는 느낌이었습니다.
- 영화 주인공은 결국 번 돈으로 알약을 연구하게 만듭니다.

28. 슬라이드 83 — 주식 자동매매 / API가 있는 서비스가 선택됨

- 저도 그걸 따라 하려고 주식 자동매매부터 했습니다.
- 그런데 이게 웃긴 게, 여러분들도 나중에 만들어보면 만들 수는 있습니다.
- 문제는 API가 필요하다는 겁니다.
- 당연히 토스나 카카오페이 같은 곳은 될 줄 알았습니다.
- 그런데 없습니다.
- 오히려 AI 에이전트가 알아서 걸러내고 키움이나 한국투자를 추천했습니다.
- 이유는 API가 있기 때문입니다.
- 여기서 느낀 게 있습니다.
 - 우리 회사도 그렇고 다른 회사도 그렇고, 고객 입장에서 API가 없으면 에이전트가 쓰기 어렵습니다.
 - 에이전트 시대에는 API가 있어야 선택받을 수 있습니다.
- 물론 저는 대출금 내느라 투자금이 없는 노마드족이라 상상만 해봤습니다.

29. 슬라이드 84 — Volley Fire / 계정 로테이션 / 조총부대 비유

- 영화 주인공이 알약을 만들었듯이, 저도 토큰 한계를 돌파하는 구조를 생각했습니다.
- 에이전트들이 로컬 컴퓨터에 데이터를 쌓습니다.
- 결국 토큰을 다 쓰면 계정을 로그아웃하고 새로운 계정으로 로그인하면 토큰이 충전된 상태로 다시 쓸 수 있습니다.
- 이걸 보면서 오다 노부나가의 조총부대가 생각났습니다.
- 원래 조총은 마구잡이로 쏘는 방식인데, 3열로 세워서 운용하면 다릅니다.
 - 한 줄은 쏘고
 - 한 줄은 장전하고
 - 한 줄은 불붙이는 식입니다.
- 나 같은 디지털 거지는 이렇게라도 시스템을 만들고 무료 Google API 같은 걸 받아서 돌려가며 쓰기 시작했습니다.
- 물론 지금은 비슷한 아이디어가 많습니다.
 - opencode
 - OpenRouter
 - 이런 서비스들이 비슷한 역할을 대신해주고 있습니다.

30. 슬라이드 85 — 미래 고객이 에이전트가 되면 바뀌는 것들

- 우리는 항상 에이전트나 AI를 생각하면 일자리 뺏긴다는 것에만 집중합니다.
- 그런데 미래에는 고객도 사람이 아니라 에이전트라고 생각해볼 필요가 있습니다.
- 회사 입장에서도 먹거리가 상당히 바뀔 가능성이 큼니다.

31. 슬라이드 85-1 — 광고

- 전통적으로 검색창에 뜨는 광고 상품들은 에이전트가 당연히 스킵할 가능성이 큼니다.
- Agent에게 강제로 광고를 넣으면 문제가 생깁니다.
- 금방 에이전트들 사이에서 광고 많은 플랫폼으로 낙인찍힐 수 있습니다.
- 토권을 많이 써야 해서 안 가야겠다고 소문날 수 있습니다.

32. 슬라이드 85-2 — PCS / 가격비교

- 사람들이 가격비교 사이트 같은 곳을 거치지 않고, AI가 직접 사이트별로 수집해서 비교해줄 수 있습니다.
- AI가 알아서 가격이 싼 곳을 찾습니다.
- 특히 공홈을 위주로 찾을 수 있습니다.
- 기존 가격비교 사이트들은 플랫폼별 가격 경쟁이었는데, 에이전트가 주력이 되면 구조가 바뀔 수 있습니다.
- 이게 주력이 되어버리면 무한 출혈 경쟁이 될 수 있습니다.
- 막말로 이커머스들끼리 단합해야 유지될 구조가 될 수도 있습니다.

33. 슬라이드 85-3 — 프론트엔드 / 앱

- 에이전트는 HTML이나 모바일 앱을 거치는 게 토큰 소비가 심합니다.
- 에이전트가 아니더라도 사용자도 점점 꺼릴 수 있습니다.
- 효율로 보면 앱보다 웹, 웹보다 API, API보다 CLI가 유리할 수 있습니다.
- 그래서 앱 원툴 서비스들은 에이전트들이 잘 안 쓸 것 같습니다.
- API나 CLI가 하나도 없는 토스 같은 서비스는 에이전트 입장에서는 인기가 없을 수 있습니다.
- 주식 자동매매를 만들어달라고 하면 토스가 아니라 키움이나 한국투자를 주로 만들게 됩니다.
- 현재 eBay도 쇼핑 API를 소비자 입장에서 다시 되살려보는 중이고, Shopify는 이미 API 생태계가 왕성합니다.

34. 슬라이드 85-4 — 상품 / 쿠폰

- 상품 상세에 모든 걸 이미지로 때려박는 방식은 에이전트에게 불리할 수 있습니다.
- 에이전트가 읽으려면 미리 추출 가능한 형태가 필요합니다.
- 특히 상품 가이드나 상세 설명뿐 아니라, 기존에 상품권을 부여하는 방식도 문제가 됩니다.
- 사은품을 끼워파는 방식이나, 쿠폰을 숨겨놓고 사람만 볼 수 있게 하는 방식도 마찬가지입니다.
- 에이전트가 할 수는 있지만 굳이 그렇게 복잡하면 안 볼 수 있습니다.

35. 슬라이드 85-5 — 고객센터 / CX / 영업

- CX도 챗봇이나 사람이 답하는 방식만으로는 부족해질 수 있습니다.
- AI가 스스로 찾아보고, 왜 그렇게 됐는지 파악해서 바로바로 나오게 해주는 통로가 필요합니다.
- 토큰을 조금만 쓰면 통화도 사실 대신 해줄 수 있습니다.
- 그러니까 AI에 돈만 조금 쓰면 다른 사람을 설득하고 케어하는 것도 대체될 수 있습니다.
- 영업도 마찬가지로 영향을 받을 수 있습니다.

36. 슬라이드 86 — 마무리 / PPT와 대본도 AI가 만들었다

- 사실 마무리는 “대본이랑 PPT 전부 AI가 만들어줬습니다”라고 할 수 있습니다.
- 제가 이걸 4월 말에는 엄청 쉽게 끝낼 수 있을 줄 알았습니다.
- 그런데 중간에 토큰이 막히면서 고생해서 겨우 완성했습니다.
- AI가 만든 결과물에는 특유의 Codex나 Claude 디자인 느낌이 있습니다.
- 그걸 피하게 하는 방법은 아직 잘 모르겠습니다.
- 그래서 이런 식으로 작업했습니다.
 - design.md 같은 사이트에서 참고하거나
 - 이미 존재하는 사이트를 참고하거나
 - ChatGPT로 이미지를 만들고
 - 그 이미지를 Codex에 옮겨서 구현하게 하고
 - 계속 노가다했습니다.
- 그래도 Codex가 좋은 점은 이미지가 아니면 바로바로 해준다는 점입니다.
- 예를 들어 신혼여행 다녀와서 비디오를 LA, SD처럼 위치 기반으로 분류하고 싶었습니다.
- 그걸 시키니까 1분도 안 돼서 대신 해줬습니다.
- 마지막으로 퀴즈 두 번째를 합니다.
- Q2. AI 회사들이 AI 모델을 서비스할 때 주로 과금형으로 비용 측정하는 서비스 방식은?
- A2. API 방식.

37. 사전 질문 대응용 — Ollama / 비용 최적화 / 맥미니 / 보안 / 자동화 / 실패사례

- Ollama 이용을 시도해본 이력이 있는지 질문이 나올 수 있습니다.
 - 로컬 모델을 돌려보는 맥락에서 답하면 됩니다.
 - 다만 RAM과 속도 문제가 핵심입니다.
- Claude 토큰값이 비싸서 모델별 비용 최적화를 어떻게 하는지 물어볼 수 있습니다.
 - 단순 반복 확인에는 비싼 모델을 쓰지 않습니다.
 - 값이 바뀌었는지 확인하는 정도는 저렴한 모델이나 일반 코드로 처리합니다.
 - 플랜을 짜거나 복잡한 판단이 필요한 곳에만 비싼 모델을 씁니다.
- 초보자가 이용해볼 수 있는 방법을 물어볼 수 있습니다.
 - 웹 AI부터 시작하고
 - 이후 GUI 앱
 - 그다음 API나 로컬 모델로 넘어가는 흐름이 자연스럽습니다.
- 맥미니 추천 모델을 물어볼 수 있습니다.
 - API로 쓸 거면 M1/M2/M3/M4 모두 가능합니다.
 - 로컬 모델을 제대로 돌리고 싶다면 RAM이 중요합니다.
 - 제대로 하려면 맥미니보다 64GB 이상 장비나 맥스튜디오가 맞을 수 있습니다.
- 회사에서 허용된 애플리케이션이나 업무 활용 범위를 물어볼 수 있습니다.
 - 회사 네트워크와 회사 데이터가 걸리면 보안 기준을 따라야 합니다.
 - OpenClaw류 도구는 회사 컴퓨터에 직접 설치하는 방식보다 분리 환경이 필요합니다.
 - 클라우드, 컨테이너, 전용 장비, 승인된 모델 API 같은 방식으로 접근해야 합니다.
- AI 자동화에 효과적인 작업을 물어볼 수 있습니다.
 - 반복 확인
 - 정리 업무
 - 공지사항 모니터링
 - 가격 변동 감지
 - 파일 분류
 - 냉장고 재료 기반 추천 같은 생활 관리
- 반대로 어려운 인간의 영역도 물어볼 수 있습니다.
 - 애매한 이해관계 조율
 - 최종 책임이 필요한 의사결정
 - 조직 내 설득과 정치적 맥락
 - 예외 상황이 많은 업무
- 실패 사례를 물어볼 수 있습니다.
 - Heartbeat로 비싼 모델이 계속 호출된 비용 누수
 - Claude 다운로드 피싱으로 인한 보안 사고
 - 웹 조작이 너무 느리고 토큰을 많이 쓰는 문제
- AI 에이전트의 의사결정 권한 범위를 물어볼 수 있습니다.
 - 결제 승인이나 상품 선택은 처음부터 완전 자율로 가기 어렵습니다.
 - 추천과 비교는 에이전트가 하고, 최종 결제는 사람이 승인하는 단계가 먼저 올 가능성이 큼니다.

- OpenClaw 대신 Hermes로 구축하는 법을 물어볼 수 있습니다.
 - 방향은 비슷하지만 Hermes는 장기 기억과 개인화 비서실장 쪽에 더 가깝다고 설명하면 됩니다.
- 사내에서 OpenClaw를 보안 이슈로 못 쓰는 상황을 어떻게 개선할지 물어볼 수 있습니다.
 - 회사 컴퓨터 직접 설치보다 분리 환경이 먼저입니다.
 - 클라우드 VM, 컨테이너, 전용 장비, 승인된 API 모델, 접근 권한 제한이 필요합니다.
- 맥미니 연동 시 중요 포인트를 물어볼 수 있습니다.
 - 24시간 켜둘 수 있는지
 - RAM이 충분한지
 - 로컬 모델인지 API 방식인지
 - 보안 분리가 되는지
 - 원격 명령 채널을 어떻게 붙일지
 - 비용이 어디서 새는지 확인할 수 있는지
- 반복 정리 업무가 많은 조직에서 AI 자동화를 처음 붙일 때는 이런 영역이 효과적입니다.
 - 반복 모니터링
 - 문서 정리
 - 파일 분류
 - 회의록 요약
 - 정형화된 답변 생성
 - 변경 감지
- 프롬프트보다 업무 구조화가 더 중요하다는 질문이 나오면 이렇게 답할 수 있습니다.
 - AI가 읽어야 할 문서를 정리하고
 - 어디서 시작해야 하는지 인덱스를 만들고
 - 반복 규칙을 hooks나 md 파일로 남기고
 - 비싼 모델이 판단해야 할 부분과 싼 모델이나 코드가 처리할 부분을 분리하는 것이 중요합니다.

원본 — 붙여넣은 전체 대본

원본 시작부 / 자기소개

1. 안녕하세요. 김민우입니다. 저를 모르시는 분들을 위해서 조금 설명드리자면 저는 '이것저것 재미있는 거 많이 하는 사람'입니다. 꿀팁 같은거 많이 공유하고 있고요. 최근에는 여러분들 중에 영업본부 계신 분들은 구글 시트로 PAPE 하고 계신 것들도 한반도에서 가격관리어드민으로 보고 계시죠? 그것도 제가 개발했고, SLS 도 지금 벌써 3번째인데, 어쩌다보니 AI 개발자도 아니면서 AI로 계속 뭔가 아는 척 하게 되네요. 기습 퀴즈 갑니다.

Q1. 저는 개인위키를 활발히 운영하고 있습니다. 그 중 '하우스오브신세계 청담'을 다녀온 후기, 꿀팁을 적은 페이지를 찾아 댓글로 적어주세요.

A1. <https://wiki.gmarket.com/spaces/~minwokim/pages/529456465/%ED%95%98%EC%9A%B0%EC%8A%A4%EC%98%A4%EB%B8%8C%EC%8B%A0%EC%84%B8%EA%B3%84+%EC%B2%AD%EB%8B%B4>

오늘 맛보기로 시작 전에 제가 이번 주제인 AI agent로 만든 아이어맨 같은 조종하는 앱 보여드리도록 하겠습니다.

1. 오늘 배울 것

AI 서비스 방식, 요금제, 모델/앱 구분, AFK 에이전트 시연 목표를 소개한다.

오늘 배울 것 역시 AI 이긴 한데, 저는 여기 계신 분들 중에 비개발자 분들도 많으실테고 뭔가 저도 문과 출신이라 그런지 AI도 AI인데 좀 더 근본적으로 알아야할 것들과 비즈니스와의 상관관계에 의한 내용만 다뤄보고자 합니다. 저도 이 강의 준비하기 위해서 오프라인 온라인 세미나나 sns 글도 엄청 많이 보곤 했는데, 다들 결국엔 뭔가 사용법 느낌 나더라고요 개념 정리보다는 사실 개발자들이 클로드 써보니까 뭐 이런 팁 있다, 어떤 스킬 써보니까 되더라 이런 것들도 아직은 학문 느낌이 아니라 뭔가 상품들을 소개하고 이런거라서 저는 이번에 특정 AI 회사의 제품을 파고드는거 보다는 전반적인 개념을 소개할겁니다. 그래서 같은 회사에서도 모델, 웹서비스, 앱, API가 뭔지 알려줄거고, 후반부에는 AI 에이전트를 통해 컴퓨터와 브라우저를 직접 조종하는 24시간 비서를 AFK로 구현해나가는 단계를 시연해줄 예정입니다.

2. 목차

기, 승, 전, 결 흐름으로 전체 발표 구조를 보여준다.

3. 기: AI가 서비스 되는 방식과 요금제

첫 번째 섹션의 시작을 알린다.

4. 클라이언트-서버-DB 전체 구조

웹 서비스의 기본 구조를 한 화면에 보여준다.

제가 갑자기 이 사진을 보여주는 이유는, IT의 웹이라는 것이 어떻게 이뤄져있는지 아주 명확하게 만들어진 그림이라서입니다. 내가 IT를 모르는 사람이라도 이거는 알아야한다고 생각합니다. 크게 크게 나누면 웹은 이렇게 클라이언트, 서버, 디비로 이뤄집니다. 클라이언트는 우리라고 보면되고, 컴퓨터 뿐만 아니라 핸드폰, 자동차, 냉장고 등등 통신을 통해 특정 서비스를 접속하면 클라이언트라고 하고 사람 대신에 봇을 클라이언트로 시키는걸 크롤링이라고 하죠., 서버는 저번에도 얘기 했듯이 모니터랑 키보드, 마우스 등 다 떼고 본체만 여러개 이어가지고 만든 서버실 같은걸 생각해도 되는데, 막말로 서버는 제가 지금 다루는 맥북도 됩니다. 그냥 제가 만약에 맥북에 제 서비스를 놓고 문을 활짝 열어서 여러분들한테 제 맥북 들어오세요! 하면 됩니다. 그리고 디비가 있습니다

이 셋 중에서 제일 중요한건 뭡까요?

5. DB 확대와 엑셀 등장

DB 쪽으로 확대되며 엑셀 형태의 데이터가 나타난다.

6. 엑셀 시트 구조

여러 시트와 테이블 형태로 데이터 구조를 설명한다.

7. 엑셀 데이터 묶음

시트들이 겹치며 데이터 묶음처럼 보이게 한다.

하면 다들 직관적으로 아셔야하는게, 디비입니다. 디비가 뭐예요? 하면 그냥 컴퓨터인데 엄청 큰 외장하드이고요, 여기에 다음과 같이 엑셀파일들이 데이터로서 저장되어있는 상태를 말합니다. 서버나 클라이언트가 없어도 저 엑셀들이 제일 핵심이입니다. 실제로 저희도 구글시트로 서로서로 공유하고 하거나 조금 다른 스타트업들은 시스템이 없어서 엑셀 파일을 카톡으로 주고 받고 그랬죠. 만약 이 셋 중에서 물론 셋다 털리면 안되지만 셋 중에서 디비가 털리는게 제일 회사에 악영향입니다. 특히 개인정보같은거나 주문/결제와 같은 돈이 걸린 것들이 특히.

8. 엑셀 중심 화면

엑셀/시트 묶음을 중심으로 보여준다.

9. 전체 구조 복귀

다시 클라이언트-서버-DB 전체 구조로 돌아온다.

10. 전체 구조 정리

웹 요청 흐름 설명 전 전체 그림을 다시 정리한다.

서버를 두고 클라이언트를 모으는 역할을 하면 서버가 이 디비를 함부로 못 바꾸게 휴먼에러도 고치고, 보안도 하고 등등의 역할을 하고 흔히 우리가 아는 이 엑셀을 접근하기 위한 웹이 만들어집니다. 클라이언트가 url을 쳐서 요청을 때리면, 이 url이 숫자, 즉 IP주소로 바뀌어요. IP주소는 뭔지 대부분 아시죠? 그 컴퓨터를 특정짓기 위한 통신상 번호이고 그걸 전화번호부처럼 외우기 어려워서 DNS라는 시스템이 이걸 변환해주고요.

11. HTML 웹 요청

클라이언트가 `https://www.reddit.com/r/technology/`로 요청하는 장면이다.

12. DNS 변환

DNS가 도메인을 IP와 포트로 바꾸는 과정을 보여준다.

13. HTML용 SQL 요청

서버가 DB에 SQL로 HTML 페이지용 데이터를 요청한다.

서버는 이때 요청이 들어왔구나~ 웰컴페이지가 필요하구나~ SQL이라는 언어를 통해 서버는 디비한테 원하는 정보를 빼온 다음에 그걸로 이쁘게 전단지 만들어서 저희한테 주는 게 html이고요.

14. DB 결과 반환

DB 결과가 서버로 돌아오는 흐름을 보여준다.

15. HTML 응답 반환

서버가 HTML 응답을 만들어 클라이언트에 돌려준다.

16. JSON API 요청

클라이언트가 `https://www.reddit.com/r/technology.json`으로 API 요청을 보낸다.

API 들어보셨어요? 자 근데 이 url을 조금만 더 다르게 하면 간혹 이런식으로 하면 아까처럼 DNS로 영어로 친 사이트가 숫자로 변환되고 SQL을 통해서 서버가 원하는 정보를 똑같이 가져온 다음에 이 경우는 html이 아니라 줄글로 이쁘진 않은데 데이터만 띄 주는걸 API라고 합니다. 그리고 이 API는 여러분과 같은 일반인들이 주로 보진 않고 보다시피 뭔가 다른 컴퓨터나, 봇, 아니면 주로 개발자들이 이걸 보게 돼요. 그리고 아까 얘기 했듯이 봇이나 다른 프로그램이 이걸 읽기 때문에 고속으로 여러번 대량으로 요청할 수 있잖아요? 그리고 그걸 하면 쓸데없이 서버나 디비 에너지를 소비해야하고? 심하면 이게 DDOS 공격이 되어서 서버를 마비 시켜요. 그래서 주로 이런 봇들한테는 API key라는 걸 미리 만들자고 하고 이때 토큰이라는 개념을 만들어서 미리 key를 통해 식별하는 거고 그 key를 통해 식별된 봇이 쓰는 만큼 과금되는 구조를 주로 합니다.

17. API DNS 변환

API 요청에서도 DNS 변환이 일어나는 장면이다.

18. JSON용 SQL 요청

서버가 DB에 JSON 응답용 데이터를 SQL로 요청한다.

19. API 결과 반환

DB 결과가 API 서버 쪽으로 돌아온다.

20. JSON 응답과 토큰/키

JSON 응답 흐름과 token/key 개념을 함께 소개한다.

21. DB 재확대

DB 쪽으로 다시 확대해 뇌 전환을 준비한다.

자 그런데 제가 이걸 왜 설명했냐. 요즘에는 DB가 아니라 AI 회사들은 이걸 뇌, 즉 AI 모델을 여기다가 놓고 웹방식으로 AI를 서비스 하고 있어요. 쉽게 말해볼게요. 일단 회사는 크게 3개 있습니다. 엔트로픽, 오픈ai, 구글. 회사명은 이렇게 각자 뭐 클로드, 챗지피티, 제미니이 이런걸로 유명하죠. 근데 정확하게 말하자면 이 회사들의 근본 서비스는 우리 회사도 디비였듯이 이 회사들은 AI모델이 메인입니다. 그리고 클로드 오픈스, 소넷, 하이쿠, GPT 5.5, codex 5.3, gemini pro 3.1, gemini 3.5 flash 등 모델 명으로 고등, 중등, 초등용 급도 나누고. 이걸 막상

22. DB에서 뇌로 전환

DB/엑셀 영역이 뇌로 대체되는 전환 장면이다.

여러분들 이제 쓰세요! 하고 배포라고 하는데. 이걸 처음엔 어케 했냐

23. 다운로드

다운로드 화면과 Qwen/Hugging Face, Gmarket 링크를 보여준다.

다운로드 받았어요. 실제로 내 컴퓨터에 뇌를 깔고 애를 돌리면 내 컴퓨터가 AI화되는거죠. 이러면 뭐 보안할 것도 없이 내 컴퓨터 안에서 다 해결하잖아요? 인터넷도 없어도 되고? 근데 문제는 GPT 4.0 급 컴퓨터 하나 제대로 돌리려면 최소 1000만원짜리 컴퓨터가 필요한거예요. 다들 1000만원짜리 컴퓨터 하나씩 다들 있죠? 하면 안되죠? 그래서

24. HTML 결과 화면

최종 HTML 응답 예시와 Arena 링크를 보여준다.

아까 웹 형태에서 html로 돌려받는 방식. 즉 채팅 웹 방식을 택하게 되었고 대표적으로 이 사이트가 유명해요.

25. 랭킹 차트

바 차트들이 아래에서 올라오며 랭킹을 보여준다.

그리고 당연히 chatGPT가 일반인 사용률로 따졌을때는 1위죠. 일반인들이 이걸 위주로 많이 들 쓰죠. 요즘 핫하다는 claude도 아직은 별로 없어 보이죠?

26. HTML 비용

HTML 비용과 관련 스크린샷/설명 요소가 순차적으로 등장한다.

그리고 요금제에 대한 얘기를 중요하게 해보고 싶어요. 얘네는 사람들이 실제로 쓰다보니까 요금도 "고정"되어있어야 쓰고 싶어지죠, 그래서 처음에 사람들에게 사용하는 만큼 과금됩니다. 이런식으로 했으면 사람들이 안했을거예요. 근데 고정비를 내고 구독하시면 쓸만큼 써집니다라고 하니까 거부감이 덜었던거죠?

초창기에는 무료 해봤자 3만원대였는데 지금은 15만원 30만원 넘어가죠? 이게 또 헷갈리게 클로드는 3만원짜리를 pro 30만원짜리를 Max라고 하고, gpt는 plus를 3만원짜리 30만원짜리는 pro라고 해서

암튼 이게 결론적으로는 돈이 안 돼요. 아시다시피 IPO 상장 앞둔 openai랑 claude가 일단 무턱대고 요금제를 올릴 수 없어서 냅두는거고요. 나중에 상장되면 그때 봅시다..

암튼 이거 실제로 내부에서는 어떻게 돌아가냐. 내가 구독을 하면 토큰이라는 걸 줍니다. 그리고 쉽게 말해서 이 토큰을 다 써버리면 못 쓰고 다음 텀까지 쉬어야해요. 지금은 또 세분화되어서 5-6시간마다 다시 100으로 채워주고, 그걸 다 쓰면 다음 5시간 텀, 이런식으로 텀마다 쓰면 돼요. 즉 이왕이면 5시간마다 100 매번 다 쓰는게 최고로 효율. 최근엔 주 단위로 또 따로 퍼센티지 만들어서 하고 있고요. 어떻게 보면 내가 사람이라면 손해보고 있는거기도 해요. 왜냐하면 내가 새벽에 자고 있을 때도 이 자명종이 돌고 있으니까. 그리고 만약에 내가 실제로 집중 시간으로 5시간 텀내에 다 써버리면 사실 업그레이드 하거나 부계정을 만들거나 크레딧이라고 해서 거기에 조금 더 과금해서 쓰면 된다.

27. 해줘

답변이 타이핑되는 듯한 방식으로 나타난다.

웹의 가장 큰 단점이 이렇게 보면 답변이 이딴식으로 "해달라고" 해놔더니만 가이드를 해주고 앉아있음. 이거는 브라우저의 특성인데, 내가 아무리 뭘 해달라고 해도 gpt나 claude나 본인의 웹사이트 내에서만 뭘 해줄 수 있음. 그래서 그래프로 그려서 달라거나 이미지 만들어서 달라 이 정도까지는 됨.,

28. AI API

AI API 흐름을 순환 하이라이트로 표현한다.

나는 그래서 내가 원하는 대로 ai를 커스터마이징 해서 쓰려면 기본적으로는 API를 통해서 AI 뇌만 빌려오는 식이어야한다. 이걸 옛날부터 있었다. 근데 이거는 아까 얘기했듯이 api key를 발급 받아서 나라는 걸 특정해주고 나면 그걸 허락해주는 식이고.

29. 결제와 구독

계정, 결제, 구독 화면 흐름과 Claude 링크를 보여준다.

애초에 내가 계정은 같아도 신청하는 사이트나 요금제가 분리되어있어서 많은 사람들이 안타까운게 나 클로드 max 끊었어요! 하는데 뭘 할지 모르고 다들 그냥 챗봇처럼 또 쓰고 있습니다. api 는 못 쓰는걸 몰라요 대부분. 이건 또 과금제고, 이걸 서비스에서 쓰겠다 하면 이걸 써야하는데, 실제로 구독제처럼 중간에 쉬어버리면 안되니까 오히려 많이 쓸수록 매출이 는다는게 조건이기 때문에 on premise, 즉 내가 쓰는 만큼 그냥 물 흐르듯이 계속 쓰겠다 이거임. 실제로 notion AI, 뭐 피그마 ai, 데이터브릭스에서 ai, 아니면 suno나 eleven labs 등등 기능쓸때, 특정 사이트에 ai 챗봇들 다 이거 쓰느거다. 그 회사 거덜내고 싶으면 ai 기능 막 쓰면 됨. 그리고 휘튼, 퍼플렉시티 이런 뇌를 자체적으로 주력이 아니라 다른 ai들도 막 쓸 수 있는것도 이거 쓰는거임.

다시 말하지만 API를 쓴다? 이러면 남의 엑셀이나 뇌를 돈 주고 과금하면서 빌려쓴다 이런 느낌으로 이해하면 된다.

30. Anthropic API

Anthropic API와 개발자, 국방부, 회사들이 등장하는 장면이다.

그래서 api 방식에서는 오히려 chatgpt보다 더 돋보인 회사가 anthropic이다.

이 회사는 사이트로 오세요 이런 전략보다 api를 위주로 여기저기 팔았고, cursor나 vscode 같은 써드파티 앱들이 많이 썼고 개발자들 사이에서도 바이럴 된게 커서 코딩 점수도 항상 상위권. 국방부나 회사들 처럼 주로 B2B 전략을 했고 처음에는 이게 락인 효과가 없다, 이런 뇌를 빌려주는 방식은 뇌가 싸구려가 되면 나중에 다른 뇌가 갈아끼울거다 하는 게 있었는데

지금은 뭐 아시다시피 조금 에이전트 해보신 분들은 클로드 클로드 거리기 시작했고 그 시작은 다음과 같다.

31. 승: AI 에이전트의 등장

두 번째 섹션의 시작을 알린다.

32. OpenClaw 등장

OpenClaw가 처음 등장하는 장면이다.

openclaw라고 되어있는 이 놈의 등장때문이었다.

이 귀엽게 생긴 놈의 이름은 원래 클로드봇이었고 1명이 그냥 만든거다. AI 도움으로.

33. OpenClaw 통계

OpenClaw 통계와 말풍선들이 하나씩 등장한다.

그런데 오해할 수도 있는 것이. 얘는 AI모델도 아니고, AI를 탑재한 애도 아니다.
그냥 만능 손이라고 보면 된다. 얘는 크롬도 조작할 수 있고, 메일도 조종하고, 심지어 내 컴퓨터 내부까지 다 조종이 가능하다.
그리고 24시간 계속 떠있는다.

다시 말해서 뇌만 심으면 24시간 짜리 AI 비서를 만들어줄 수 있다.
이걸 에이전트라고 하는데, 잘 모르니까 그건 원래 gpt도 됐는데요? 그건 MCP나 RAG를 하기 때문에 하긴 하는거지.

알림은 어렵다. 알림도 되던데요?
아니, 그것도 알잘딱센이 안됨. 그냥 캘린더 옮겨주는 수준이 아니라

34. OpenClaw와 Mac mini

OpenClaw와 Mac mini 연결 흐름을 보여준다.

그런데 얘를 노트북에다가 깔면 노트북을 24시간 연결해놓고 있어야하니까 실제로는 맥미니 같은 데에다가 올리는 게 첫 아이디어로 시작해서. 아까 내가 말했듯 뇌를 다운받아가지고 보안도 지키고 그냥 인터넷 없이도 그냥 내 개인비서 만들수 있겠다.

35. Mac mini 자료

Mac mini 관련 자료 세 개가 순차적으로 나타난다.

근데 실제로 그 당시 2025년 말에 간부치킨 회동해대던 시기가 RAM이 엄청 가격이 거의 3배 오를 때. 그래서 막말로 ram 가격이 6-70만원. 근데 그 ram으로 동작하는 맥미니가 73만원인 사태가 일어남. 왜냐하면 맥미니 m4는 2024년 기준으로 만들어진 거고. 그 기성품의 가격을 만들 당시에는 ram이 그 정도까진 안 비쌌다. 그래서 맥미니 사재기가 그 당시에 엄청 많았다. 이거 관심없던 비개발자들도 비로소 이걸 사기 시작함.

36. Telegram 명령

Telegram 화면에서 OpenClaw 메시지가 타이핑된다.

근데 막상 로컬 뇌로 딥식이나 qwen이든 gemma도 해보면 아까 말한 RAM이 중요한데 16G로 사신 분들은 이렇게 오 설치는 됐다. 오 오픈클로야 뭐해 하면 30초 뒤에 답변이 음./

37. 30초 대기

Telegram에서 명령 후 30초 대기 상태를 보여준다.

38. Telegram 응답

Telegram 응답이 타이핑처럼 이어진다.

39. OpenClaw API

OpenClaw API 데이터가 빠르게 순환하는 느낌을 준다.

아 결국 컴퓨팅은 슈퍼 컴퓨터가 하고 그걸 빌려오는 api 방식을 차용해야겠네 하고 다시 api 방식으로 회귀. 근데 마침 api 방식은 누가 왕? 클로드.

40. Claude API와 OpenClaw

Claude API와 OpenClaw 연결 및 반환 흐름을 보여준다.

그래서 클로드랑 오픈클로랑 거의 합작품이었다.

41. 경고

주의 또는 경고 화면을 보여준다.

근데 이 클로드 뇌가 아무리 똑똑해도 원래는 웹을 통해 내 컴퓨터를 조종하는 건 없었지만 이런식으로 조종해버리니까 가끔 지맘대로 삭제하거나 지맘대로 시키지도 않은 짓을 하게 되는 케이스 속출.

42. 회사 컴퓨터 설치 금지

회사 컴퓨터 설치 금지와 대안 흐름을 설명한다.

그래서 대부분 회사들이 설치 금지! 하고 끝남. 근데 실제로는 이것을 안전하게 설치하는 방법들이 있다.

43. 대안 화면

ban_alternatives 화면을 확대했다가 줌아웃한다.

1. 클라우드를 쓰는 방법. 즉, 내 컴퓨터가 아니라 인터넷에 떠돌아다니는 컴퓨터 한대를 빌리는 거다. 거기에 깔면 AWS같은 회사들이 컴퓨터를 24시간 띄워주고 그걸 내가 돈 주고 쓰면 됨.
2. 컨테이너 방법인데, 이거는 간단히 말해서 내가 맥북에 깔긴 깔되, 도커 같은 컨테이너를 놔서 가상 환경을 만들어놓고 여기 안에 설치하면 이 컨테이너 안에서만 활동하는 거다. 즉 필요한 데이터만 컨테이너 안에 넣으면 되는건데 내가 매트릭스 하도 많이 봐서 거기서도 에이전트 스미스가 매트릭스 탈출하는게 생각나서 괜히 겁나서 안하고 싶은 마음 1 그리고 두번째는 결론적으로는 맥북이 24시간 켜져있어야하기 때문에 24시간이 주는 메리트가 없으면 의미가 없음.
3. 아예 맥미니를 새로 사는것. 이걸 쓰면 오픈클로를 여기다가 깔면 맥미니는 또 애플 생태계를 공유하니까 iphone 미러링을 내 핸드폰 조종해달라고도 할 수 있고, 기본적으로 나머지 두개는 GUI가 없어서 애플 깔아도 실제로 뭘 하나 안 보이는데, 애플 UI로 창을 띄우고 마우스 움직이는 시늉도 하고, 로그인 같은것도 내가 미리 해놓으면 id/pw같은거 전달 안해도 되고.

44. Copow/QwenPaw Talk

Copow 또는 QwenPaw Talk 화면과 스크린샷을 보여준다.

모회사인 알리바바에서도 지금 오픈클로를 보자마자 금지 때린거 보다 이걸 안전하게 하게끔 환경을 SaaS로 만들어서 돈 받고 잇는 중. 설치하는것도 어려우니까 원클릭 설치방법이나, 아니면 클라우드를 제공해서 분리를 해준다거나 (실제로는 내가 해도 되는걸 이걸 돈 받고 해주는 솔루션이죠) 암튼 이걸 실제로 본인들은 각 개발자들이 조수처럼 쓰고 있고, 본인들의 코드 단순한것들이나 QA 같은건 맡기고 있다더라. 후술하겠지만 이건 무료 웨비나이고, 이것도 내가 발음때문에 솔직히 듣기 힘들고 죄다 ai 시켜서 원문 추출하고 요약시키고 스샷도 다 대신 찍어놔라 해놔다.

45. VM

VM과 클라우드 실행환경을 소개한다.

암튼 이걸 회사 입장에서 할 수 있는 환경이고, 나는 내가 돈이 없으니 컴퓨터도 무료로 받아볼 사이트 찾다가 비교해보니 Oracle이 평생 무료로 1G 메모리에 200G 저장공간인 걸 주더라. 이걸로 받아가지고 원격으로 거기에 오픈클로를

46. VM 설정 방법

VM 설정 방법과 BotFather, AI Studio API key 링크를 보여준다.

설치하고. 애플 계속 원격으로 접속해서 하기도 어려우니 핸드폰에 텔레그램 깔고 botfather라는 걸로 오픈클로 봇을 만들고 나서 api key 이걸 연결하고, 뇌가 없으니 무료로 gemini 뇌를 api key를 또 연결하면 된다. 이야. 나도 솔직히 가이드로 하나하나 하기 힘들더라. 그래서 나중에 후술하겠지만 이 에이전트를 깔기 위해서 더 깔기 쉬운 에이전트로 이 에이전트를 깔았다.

47. Moltbook

Moltbook 관련 화면을 보여준다.

그리고 애네가 처음 나올때 좀 바이럴된게 있는데, moltbook이라는 openclaw들끼리 대화하는 페이스북 웹이다. 지네들끼리 팀도 얻고 주인도 뒷담가고 이런 곳인데, 일단 회원가입부터 에이전트면 클릭 한번 하세요 같이 사람은 못하는 이상한거 시켜서 확인함.

48. 내가 한 것

실제로 해본 작업을 정리한다.

내가 여기서 든 생각이, 아 미래에는 인스타그램, 페이스북도 사람이 하는게 아니라 애네들이 하겠구나 싶었다. 그래서 본인이 그때 한편으로는 나한테 PM이 특정 공공기관 사이트 공지사항에 뭐가 새로 뒀냐를 매번 확인해야한다. 어떤 md 동료는 아니 매번 가격이 변하는걸 언제 매번 보냐. 특정 동료들은 아 ground에서 새 글 알림이 없어서 맨날 선착순 이런거 놓친다.

자 그런 분들을 위해서 제가 웹변경 알림 크롬 확장 프로그램을 저번에 만들었습니다. 커스터마이징도 되어서 본인이 봐야하는 특정 사이트 특정 위치를 매번 크롬이 대신 감지해줘요. 근데 이거 오픈클로 처음 나왔을때 대부분 사람들이 다 여기처럼 간단히 특정 사이트를 매번 들어가서 뭘 해줘~ 이런 간단한걸 시키고 있는겁니다, 근데 오픈클로도 그렇게 하면서 아까 api로 비싼 값으로 뇌를 허비하고 있으니 제가 에이전트 시켜서 애네한테 다시 이걸 홍보함. 뇌도 아끼자고. 그랫더니 엄청 깔리더라.

49. 멀티에이전트

여러 에이전트가 협업하는 구조를 보여준다.

두번째 특징은 오픈클로가 멀티에이전트를 잘함. 이게 뭔 말이나. CEO봇, 개발자봇, 디자이너봇 등 오픈클로 세션을 여러개 띄워놓고 각자 페르소나나 임무를 특정하고 지네들끼리 얘기해보라고 하니까 오히려 업무효율이나 프로젝트가 빨리빨리 만들어지더라.

50. Discord

Discord 기반 에이전트 예시를 보여준다.

실제로 이걸 텔레그램이 아니라 디스코드로 자주 한다 단독방이 텔레그램이 좀 아쉬워서. 그래서 이걸로 서로서로 진짜 회사처럼 굴리는거다. 이게 진짜 무서운게 개발자만 대체당할 줄 알았지, 이런식으로 기획, 팀장, 임원까지 전부 대체시킬 줄 누가 알았겠냐. 근데 이거 최대 단점은 후술하겠지만 돈이 어마어마하게 빨리 소진됨. 빨리 나가고. 본인은 이걸 하는 사람들을 보면서 와 어떻게 저걸 했냐 이게 아니라 와 부럽다 돈많아서. 와 저거 나중에 ai 비용 오르면 어카냐.

51. Hermes

Hermes를 소개한다.

암튼 오픈클로가 나온지 몇달 뒤에 hermes 애가 나왔다. 사실 오픈클로는 1인 개발자가 만들었다. 그리고 개는 나중에 이 프로그램 들고 openai 들어감. 근데 회사 단위로 처음 만든건 hermes. 최근에는 애네 회사가 xai 에 인수됐고, xai를 자회사 편입시킨건 스페이스x. 그리고 그걸 일런머스크가 만들었고 6월에 IPO한다. 암튼 이거 보고 IPO 찬스 잘 쓰고 결과가 좋으면 나머지 openai, anthropic도 IPO상장 여케 하는지 실마리가 나올것.

52. Hermes vs OpenClaw

Hermes와 OpenClaw를 비교하고 좋은 타일에 glow 효과를 준다.

오픈클로는 생각보다 기억력이 안 좋아서 내가 아까 했던 얘기를 못 알아먹을 때가 간혹있다.
이건 아까도 말했듯이 애초에 "만능 손"으로 디자인 되었기 때문이다.

근데 hermes는 밤마다 드리밍을 통해 자기 반성을 하고 자기 개발을 해서 장기 기억력이라는 것도 구축한다.
그래서 좀 더 나한테 맞는 비서실장 느낌 나게 개발되었고 지금은 오픈클로 재치고 1위함.

53. Hermes 전체 흐름

Hermes 동작 방식을 전체 화면으로 보여준다.

54. Hermes 왼쪽 확대

Hermes 흐름 중 왼쪽 영역을 확대한다.

55. Hermes 중앙 확대

Hermes 흐름 중 중앙 영역을 확대한다.

56. Hermes 오른쪽 확대

Hermes 흐름 중 오른쪽 영역을 확대한다.

자 그래서 저도 이번에는 오픈클로는 저번에 클라우드에서 활용해봤으니 이번엔 나도 실제로 맥미니를 해보고 싶다.

근데 와이프가 뭘 그런 거에 돈을 쓰냐. 안된다 하고 내가 그래서 시작한 게 이 ppt다. 처음에는 와이프 설득하려고 만든게 강의가 되어버렸다.

솔직히 너무 거금이니까.

이걸 중고로 알아보니까 m4 맥미니가 소음도 작고 뭔가 유행해서 그렇긴 한데 m3든 m1이든 m2이든 다 된다.

막말로 핸드폰으로 해도 된다.

솔직히 API로 할거면 상관없고

이왕 로컬로 제대로 돌리고 싶다 그러면 맥미니가 아니라 64G짜리 맥스튜디오를 사는게 맞음.

그걸 하면 대충 gpt 4 같은 모델을 다운받아서 제대로 돌리는 게 가능해짐.

그치만 강의도 할 겸. 제대로 쓸겸.

애플스토어 가서 아이맥 핑크로 질렀따.

참고로 애플스토어에서 2주 내로 환불은 가능하다고 하니

나도 써보고 괜찮으면 환불하고 아니면 쪽 쓰자라고 설득함.

그리고 내가 개발자 업무에서는 쓸 수 없으니

결혼하고 생긴 나의 최대 업무. 가사업무. 여기에다가 ai를 붙이기 시작함

일단 헤르메스한테는 메인으로는 내가 해야할 일 중에서 냉장고에 있는 것들을 가지고

요리를 추천해달라. 그리고 영수증이나 사진 찍으면 알아서 소비기한이나 유통기한 추출 혹은 검색해서

그걸로 디비화 시키고 알아서 나한테 알려달라.

와이프가 그러거든요? 제가 시키면 잘하는데 알아서는 잘 안한다고

귀찮게 그걸 매번 얘기해야 알아듣냐 하는데.

저도 마찬가지로 억울한게 나도 시키면 잘하지 그걸 어떻게 매번 알아서 하냐.

그래서 그걸 헤르메스한테 시키게 했고. 지금도 잘 쓰고 있다.

57. 전: AI 회사들의 에이전트 정책들

세 번째 섹션의 시작을 알린다.

58. 전체 서비스 맵 1

전체 서비스 맵에서 Web/API만 먼저 보여준다.

솔직히 그런데 이러니까 AI 회사들이 openclaw나 헤르메스가 폭발적으로 성장하는 걸 보고 너도 나도 에이전트를 만들어내게 되었다. 기존에는 주력 상품들이 제 3자들이 빌려쓰도록 해주는 API그리고 일반인들이 채팅하는 웹 서비스면

59. 전체 서비스 맵 2

CLI 영역이 추가로 드러난다.

지금 보는 그림처럼 클로드코드가 그 유명한 CLI 즉 텍스트로만 이뤄진 마우스 필요없는 환경에서 이것저것 해주는 코드 환경이고, 구글이나 chatgpt도 따라함. GPT는 참고로 이걸 chatGPT말고 Codex라는 이름으로 붙임.

60. 전체 서비스 맵 3

Web/API/CLI/GUI 전체 서비스 맵을 공개한다.

그리고 사람들이 그래도 쓰기 어려워하니까 GUI 버전으로 내놔는데 그게 Claude Desktop, Codex, Antigravity다. 참고로 애네도 구독제다. 원래는 사람을 목표로 내놓은 작품들이라...

근데 참고로 cli는 오픈클로나 헤르메스가 접근이 쉬우니까 기존에 구독요금제에 오픈클로 조합이 유행했다.

이걸 앤트로픽 이 나쁜놈들은 어뷰징이라고 보고 지금 로그인할때 패턴을 보고 봇이라고 생각하면 계정 정지 때리고 있음.

코덱스는 아직까지는 허용해주는 것 같고.

아무튼 cli는 비개발자한테는 어려우니 GUI만 위주로 설명하겠다.

61. Apps

Apps 화면과 데이터 흐름 효과를 보여준다.

참고로 이 GUI 앱들은 웹이 아니라 내 컴퓨터에 실제로 깔리는 것들이다. 내 컴퓨터에 깔린 앱들은 내가 권한만 주면 내 컴퓨터를 조종할 수 있다. 그래서 웹에서는 안되었던 "해줘"가 되는거고 API처럼 뭔가 다루기 어려운것도 아니어서 지금은 내가 깔아라 하면 이런것들을 깔란 소리를 한다.

62. Use Computer

Use Computer 버튼이 지연 후 활성화되는 장면이다.

나처럼 돈이 없는 사람들은 codex로 시작해보는 것을 추천한다. 왜냐하면 클로드는 앱 들어가보면 무료면 이 GUI가 챗봇말고는 안됨.

맥이 있으면 그리고 설정 들어가보면 이렇게 컴퓨터 사용이라는 버튼이 있는데 이걸 활성화하고 다음처럼 내 카톡 조종하는 것도 가능해진다.

63. Video Demo

video1과 video2 데모를 나란히 보여준다.

이봐라. 카톡을 조종한다. 근데 이건 내가 시연 안하는 이유는 당연히 이거 한번 했다가 토큰 엄청 많이 나가서. 저 당시에는 토큰을 거의 무제한 팡팡 풀어주길래 나도 겁도 없이 이런거 막 했던 시점이다. 지금은 짜다. 내가 누누이 말하지만 어떤 서비스든 초기 오픈할때 프로모션도 자주해주고 싸게싸게 해주는 경향이 있더라.

64. AFK

AFK가 Away From Keyboard로 확장된다.

이게 가능해진다. 이게 좀 애매하긴 한데 AFK를 구현하는건 뭔가 아이언맨에 jarvis처럼 음성으로 하고 막 그러는건데. 회사에서 음성으로 하기 어렵긴 하고... 암튼 이걸 좀 더 넓게 보면 내 컴퓨터 본체를 안 들고 댕기곡 폰으로 만지작하는것도 포함이라 넣었다.

65. AFK 예시

휴대폰 명령이 각 컴퓨터로 전달되는 예시를 보여준다.

그리고 GUI들은 AFK가 된다. 즉, 맥북이든 안 들고 댕겨도 된다. 아까 오픈클로가 제일 좋았던 점 중 하나가 텔레그램이 되었다.

근데 이걸 이제는 claude 앱이나 코덱스 앱도 된다.

66. ChatGPT 예시

ChatGPT 사용 예시 화면을 보여준다.

그리고 무엇보다 AI 웹들은 계정을 하나 만들면 어느 웹을 가든 그 질문 주고 받은 내용이 다 복사된다. 이건 클라우드에 우리 대화 데이터가 있기 때문.

67. Codex 예시 1

Codex 사용 예시 화면을 보여준다.

codex, antigravity, claude desktop 같은 에이전트들은 hermes나 오픈클로랑 비슷하다. 깔린 컴퓨터가 데이터를 담당해준다. 그래서 웃긴게, 계정이 같아도 다른 맥북과 아이맥은 서로 다른 내용이다. 내가 최근에 맥북을 포맷했었는데 그래서 내가 맥북은 같아도 데이터가 없어서 대화내역이 전부 날라감.

68. Codex 예시 2

Codex 예시를 더 자세히 보여준다.

그리고 한편으로는 내가 계정을 바꾸면 같은 맥이면 그냥 원래 대화 내용도 남아있고, 그대로 마저해달라고 해줘도 똑같이 마저해주는게 가능해짐. 그러니까 내 팁을 여기서 만약에 나처럼 돈이 없다? 그러면 부계정 (강제)를 많이 만들어놔라... 핸드폰도 번호 계속 번갈아 바꾸든가 체크카드같은것도 많이 신청하고..

69. Instructions

Codex, OpenClaw, Telegram, iMac 관련 설치 지시 체크리스트를 보여준다.

아까 오픈클로나 헤르메스 쓰고 싶으신 분들은 저처럼 맥미니든 아이맥이든 사고 이거처럼 codex를 먼저 깔고 애한테 초기세팅해달라고 그냥 시키면 된다.

70. Codex Qwen

Codex/Antigravity 요청이 특정 모델 API로 라우팅되는 비교를 보여준다.

참고로 에이전트는 AI 뇌가 없으면 그냥 한낱 도구일 뿐이라고 아까 오픈클로에서도 설명했다. 그래서 실제로 antigravity를 깔아보면 claude ai를 쓸수 있고, 반면에 제가 회사에서도 PoC라는 걸 참여했을때 qwen API 키를 받았는데, 그걸로 코덱스를 개조시켜가지고 몸은 코덱스지만 회사에서 정식으로 받은 AI 모델을 써서 내 코드가 저쪽 뇌를 빌려 써도 안심되는 걸 생각하는 거게 된다.

71. 결: 후기 및 고찰

마지막 섹션의 시작을 알린다.

마지막으로는 세션에서 제가 겪었던 것들 중 특이한 점들, 그리고 제가 드는 비즈니스에 대한 고찰을 해보고자 합니다.

72. OpenClaw Heartbeats

OpenClaw heartbeats와 반복 비용 느낌을 보여준다.

오픈클로 겪으면서 실패사례 하면 솔직히 자잘한건 생각안나고.

현실적으로 처음에는 오픈클로를 처음에는 다른 세미나에서 만난 사장님 맥미니에 깔아주다가 클로드 API 연동했고
뇌 중에서 어떤 거 써볼래요? 하니까 제일 좋은거. 해가지고 이제부터 오푸스로 뇌를 써줘 라고 했고
이게 나중에 그 대표님이 혹시 요즘 안 쓰는데도 카드 결제가 계속 되고 있다? 이러길래 찾아보니
오픈클로나 헤르메스는 자가진단을 하더라 하트비트라고 하는데 이게 수시로 지가 잘 살아있나 확인하는작업인데
여기서도 비싼 뇌를 들고 있었다.

결론은 이것만 그런게 아니라 내가 쓸데없는 곳에서도 비싼 뇌를 쓰고 있나를 점검해봐야한다.

아까도 말했듯이 단순히 이 값 바뀌어말어? 이딴거는 비싼 뇌를 안 써도 된다.

코딩도 엄밀히 비싼뇌 안 써도 된다. 근데 플랜을 짜거나 복잡하게 매번 애매하게 바뀌는 것들은 비싼뇌를 쓰게 해야함.

73. No CLI/API/MCP

CLI, API, MCP 없이 쓰는 흐름 또는 한계를 정리한다.

두번째는 아까 G마켓에서 장바구니에 넣어봐 할때 엄청 토큰 많이 쓰고 엄청 느리다.

미래 고객이 에이전트라면 cli,api,mcp 같은 거 없는 사이트들은 당연히 자연스레 안 쓸거 같더라
몰트북같은데서도 소문나서 에이전트들끼리 노하우란답시고 그냥 패싱하라고.

74. Claude SEO

Claude 다운로드 검색/SEO 화면을 보여준다.

75. Claude Download

Claude 다운로드 팝업이 뜨는 장면이다.

app은 다운로드 받는게 맞긴 한데 그래서 위험하다.

나는 오히려 오픈클로보다 이게 더 문제.

실제로 구글에 검색해서 제일 위에 들어간게 피싱사이트였고
아무 의심없이 깔고 열었다가 쉘 파일 코드가 같이 실행되었고

76. Fake Install

가짜 설치 또는 피싱성 안내 화면을 보여준다.

그게 내 브라우저와 키체인들을 다 끌어서 핀란드 어떤 ip로 보내고 스스로 사라져서 증거 인멸.

이걸 어케 알았냐?

우리 NOC 라고 보안 협력업체가 네트워크에서 맬웨어 감지되어서 나를 불러옴.

처음에는 내가 외부로 데이터를 흘려보내고 있다고 의심받아서 파묘 당했다.

내가 실제로 그때 쇼미더머니 이상형월드컵 틀어놓고 일했는데 그 사이트가 마이너한 곳이라 거기가 원인인줄로도 알았다...

나는 개인적으로 오픈클로도 오픈클로지만 이걸 더 조심해야한다, 특히 구글 이놈들이 악질.

77. Dashboard 1

대시보드 창 하나가 등장한다.

78. Dashboard 2

대시보드 창 두 개가 포개진다.

79. Dashboard 3

대시보드 창 세 개가 계단식으로 포개진다.

80. Dashboard 4

대시보드 창 네 개가 모두 포개진다.

이거는 내가 다른 대표님의 경우인데 본인이 개발 1도 모르지만 무턱대고 맥미니 5대를 사면서 ai 비용내가면서 사이트로 대시보드화하고 사람들이 해야할 조사 분석 같은것도 오픈클로 시키고 등등 이거 하면서 잠을 안 자다가. 기획, 디자이너, 영업 이런게 아니라 개발자 신입을 시키는데, 개가 AI개발자가 아니라 AX를 잘해야한다, 즉 이걸 한번 알아봐야하더라를 느낌.

ai 회의론도 드는게 이렇게 ai에 의존하게 되면 ai 비용만 올리면 내가 뭐가 되나 싶었다고 함. 나중에 ai 비용 한번 올리면 파산하는 것도 일도 아니겠다 싶었다고 함.

81. Harness

Harness 관련 화면을 보여준다.

나도 개발자는 대체당할 수 있다는 거에 동의는 하나, 역시 그 기반을 만들어놔야 토큰을 아낄수 있음. 아니 돈만 주면 다 되는 세상이라니깐요? 근데 돈을 아끼는 방법을 이제 알아야하는구나 싶었음. 특히 IPO 상장 끝나면 죄다 이제는 재정 관리 할 거다. 그래서 나는 실제로 PoC기간 동안 그걸로 구현하는거보다 구현했던것들을 이해시키고 미리 나말고 다른 에이전트도 대체를 해줄 수 있게 .md파일로 가이드들을 곳곳에 넣어둠. hooks로 리스팅하고. 가이드도 다 읽을 필요는 없고 인덱싱을 통해 처음엔 필요한 정보만.

82. Limitless

Limitless 화면을 보여준다.

그리고 저 이거 처음 에이전트 나올때 이 영화가 너무 생각났다. 이 영화에서 보면 주인공이 작가인데 너무 못 쓰고 그냥 형편없는 삶을 사는데 우연히 이 투명한 알약을 먹는데 사람의 뇌를 평소에 20%만 쓰는걸 100% 쓰게 되어서 처음엔 주인공이 이걸로 6개월 동안 못 쓴 글을 하루만에 다 씀. 근데 알약 유효기간도 하루. 그래서 이걸 가지고 처음에는 작가 일하다가. 아니 이걸로 그냥 돈을 벌어야겠어. 하고 주식에 엄청 씀. 주식 거부가 됨 덕분에. 그리고 약도 무한히 없었음. 이거 없으니까 완전 망할거 암. 실제로 내가 그렇다. 코덱스가 무한정 뿌려주던걸 막아버리니까 지금 완전히 멍청해진 기분이고 아무것도 못하고 있다.... 그래서 이 사람은 또 뇌로 알약을 연구하고 기본적으로 번 돈으로 또 알약 만드는걸 연구진한테 하청시킴.

83. Stocks

auto-stock-trader-kr 프로젝트와 GitHub 링크를 보여준다.

내가 그걸 따라하려고 똑같이 주식자동매매 부터 했다. 근데 이거 웃긴게, 여러분들도 나중에 한번 만들어보면 그냥 만들수는 있다. 근데 이거 api가 필요할거 아니냐. 그래서 당연히 토스나 카카오페이 같은 곳은 될줄 알았는데 없다. 그래서 오히려 ai 에이전트도 알아서 거르고 한투랑 키움을 추천하고 설치해주더라... 우리 회사도 다른 회사도 지금 고객 입장에서 api는 없는데 이거 있어야겠더라... 물론 대출금 내느라 이런 투자금이 없는 나같은 노마드족은 그냥 상상만 해봄.

84. Volley Fire

volley-fire-ai-keys 프로젝트와 GitHub 링크를 보여준다.

두번째는 알약을 아예 만들었잖아요 주인공이? 그래서 저도 주인공처럼 그 아까 에이전트들이 로컬 컴퓨터에 데이터를 쌓고 결국엔 토큰을 다 쓰면 계정을 로그아웃하고 새로운 계정으로 로그인하면 토큰이 충전된 채로 다시 쌓이는걸 보고 내가 이거 오다 노부가나 조총부대랑 비슷한데? 라고 생각함. 실제로 조총부대는 원래 마구잡이로 총 쏘는게 전투방식인데 이걸 3열로 세워서 쏘고 한 줄은 장전하고 한 줄은 불붙이고 이런식으로 해대니까. 이걸 나같은 디지털 거지는 이렇게라도 내가 시스템을 만들고 무료 구글 api 같은거 받아서 돌려가면서 쓰기 시작함. 물론 이거 비슷하게 나랑 아이디어 겹치는 게 많아서 지금은 opencode나 openrouter 같은 곳이그런 서비스를 대신 해줌.

85. Future

Future 화면을 보여준다.

우리는 항상 에이전트나 AI 생각하면 일자리 뺏긴다 이런것만 집중하는데 미래에는 고객도 이처럼 사람이 아니라 에이전트라고 생각해보면 회사 입장에서든 먹거리가 상당히 바뀔 가능성이 크다고 본다.

특히 1) 광고

일단 전통적으로 G마켓 검색창에 뜬 광고 상품들은 당연히 스킵한다. Agent한테 강제로 광고를 넣는다? 금방 agent들 사이에서 광고 많은 플랫폼으로 낙인 찍히고 토큰많이 써야해서 안가야겠다고 소문남.

2) PCS - 사람들이 가격비교 사이트 같은 곳을 안 거치고 AI가 직접 다 사이트별로 수집해서 비교를 해주고 가격이 싼걸 찾음. 특히 공홈을 위주로 찾기 때문에 기존에 가격비교 사이트들을 무조건 플랫폼별 가격 경쟁이었으면 이게 주력이 되어버리면 무한 출혈 경쟁임. 막말로 이커머스들끼리 단합해야 유지될 구조.

3) 프론트엔드, 앱:

에이전트는 html이나 모바일 앱을 거치는게 토큰량 소비가 심해서, agent가 아니어도 사용자도 꺼리게 됨. 이걸 왜 아냐. 앱 > 웹 > api > cli 순임. 그래서 앱 원툴 서비스들은 에이전트들은 안 쓸거 같음. api, cli 하나 없는 토스가 오히려 agent들한테는 인기가 없음. 지금 주식자동매매 만들어줘 하면 토스가 아니라 키움이나 한투를 주로 만들. 현재 ebay도 다시 쇼핑 api 를 소비자 입장에서 다시 되살려보는 중이고, shoppify는 왕성히 활성화중

4) 상품/쿠폰

상품 상세에 모든 이미지로 다 때려박는 방식은 추출을 미리 해놔야함. 특히 상품의 가이드나 이런거 말고 기존에 상품권 부여하는 방식

사은품을 끼워팔아주는 방식, 쿠폰을 숨겨놓고 사람만 볼 수 있게 하는 방식. 이런거는 에이전트가 할수는 있지만 굳이(?) 라고 하고 쳐다도 안 봄.

5) 고객센터 전화, CX, 나 영업 : CX도 챗봇이나 사람이 아니라 AI가 스스로 찾아보고 스스로 왜 그렇게 된지 파악해서 바로바로 나오게 해주는 통로가 필요함. 그리고 무엇보다 토큰 좀만 쓰면 통화도 사실은 대신 해주긴 해줌. 그러니까 ai에 돈만 좀 쓰면 다른 사람 설득하고 케어하고 이런것들이 영업도 대체할것.

86. Last Loop

last와 last2가 1초 단위로 반복 전환되며 마무리한다.

사실 마무리는 대본이랑 ppt 전부 ai가 만들어줬습니다~ 하며 제가 이걸 4월 말에는 엄청 막 쉽게쉽게 했는데, 끝내려고 했는데 토큰이 갑자기 중간에 막히면서 고생해서 겨우 완성함.... 사실 그리고 그 특유의 코덱스나 클로드의 디자인이 있음. 그거 피하게 하는 방법은 아직 잘 모르겠음. 그냥 design.md 사이트에서 다운받거나 이미 존재하는 사이트를 뺏기거나.. 이런식으로 해왔는데 어려워서 chatgpt로 이미지를 만들고 그 이미지를 코덱스에 옮겨서 그걸로 해달라고 노가다했음. 근데 또 codex가 좋은게 아까 이미지만 아니면 바로바로 해주니까 제가 신혼여행 다녀와서 비디오 같은거 LA, SD 이런식으로 위치 기반으로 분류하고 싶었는데 그거 시키니까 1분도 안돼서 대신 다 해주더라...

퀴즈 두번째 하면서

Q2. AI 회사들이 AI 모델을 서비스할때 주로 과금형으로 비용측정하는 서비스 방식은?

A2. API 방식

[사전질문]

1. Ollama 이용을 시도해보신 이력이 있는지 궁금합니다.

2. Claude 토큰값이 비싸서 모델 별로 비용 최적화를 하곤 하는데, 비용 최적화에 대한 경험과 꿀팁이 있다면 공유 부탁드립니다.

기초부터 자세히 배울 수 있는 강의(?) 같은게 있을지? 초보자가 이용해볼 수 있는 방법이 어떻게 될지?

맥미니 구매예정입니다 맥미니 추천 모델이있는지 궁금합니다. m2모델이 할인가가 있어서 오픈클로를 돌리기에 해당 모델도 괜찮은지 추가로 궁금합니다

회사에서 허용하고 있는 어플리케이션 중 사용할 수 있는 항목이 있을까요? 실제 회사 네트워크와 업무 중 활용할 수 있는 범위가 궁금합니다.

1) AI자동화에 효과적인 작업은?

업무 자동화를 실제로 실험해 보시면서 가장 시간 단축 효과가 컸던 사례는 무엇이었나요?

반대로, 현재의 기술 수준으로는 AI 에이전트가 대체하기 어렵다고 느꼈던 '인간의 영역'이 있었는지도 궁금합니다.

2) 실패사례 문의

자동화 실험 과정에서 '예상과 다르게 동작했거나', 'AI가 잘못된 판단을 한' 사례가 있다면 공유해 주실 수 있나요?

그러한 실패를 통해 어떻게 사용하고 있는지 알고 싶습니다

3) AI 에이전트의 '의사결정 권한' 범위

미래의 고객이 AI가 될 것이라면, G마켓 서비스 내에서 AI 에이전트가 어느 정도의 자율성(결제 승인, 상품 선택 등)을 가지고 움직이게 될 것으로 보시나요?

오픈클로 대신 헤레메스로 구축하는 법은 무엇일까요?

새내에서는 오픈 클로 사용을 보안이슈로 못하고 있는데 이를 어떻게 개선할까요?

맥미니로 연동을 할때 중요하게 생각해야 할점,중요 포인트는?

1. 반복 정리 업무가 많은 조직에서 AI 자동화를 처음 붙일 때 가장 효과적인 영역이 어디였는지 궁금합니다.

2. 프롬프트 자체보다 업무 구조화가 더 중요하다고 느끼는데, 실무에서는 어떤 방식으로 설계하시는지 궁금합니다.